Lecture 1, GPU & Inference Intro

Nebius Academy · Performance Engineering

13 May, 2026 · Alexey Bukhtiyarov

Compute is outgrowing memory

- Over the last 8 years: compute grew ~18×, bandwidth only ~9×.
- The FLOP-per-byte you need to keep the GPU busy keeps climbing
- New hardware alone won't save you; closing the gap between peak and achieved FLOPS is the job.

	V100 (2017)	A100 (2020)	H100 (2023)	B200 (2025)
Peak FP16 tensor-core (TFLOP/s)	~125	~312	~989	~2250
HBM bandwidth (TB/s)	0.9	2.0	3.35	8.0
FLOP you must do per byte you read	~140	~156	~295	~280

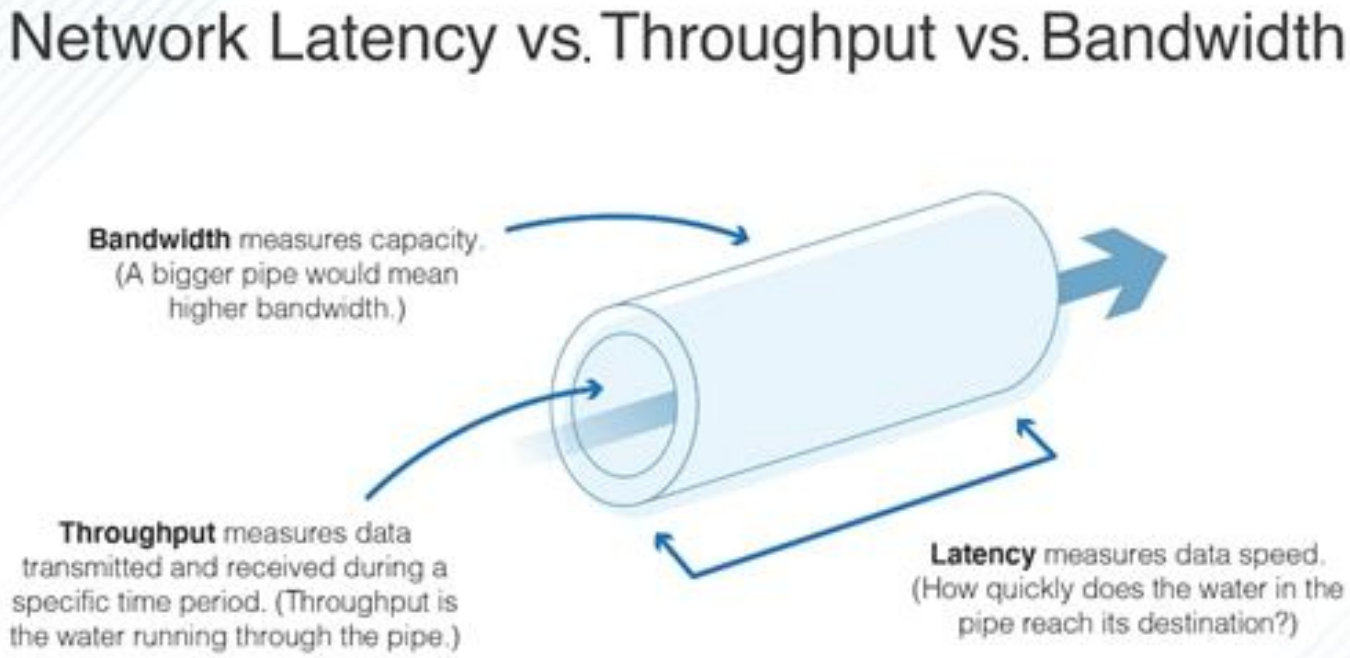
A story from practice: H100 was cheaper than A100

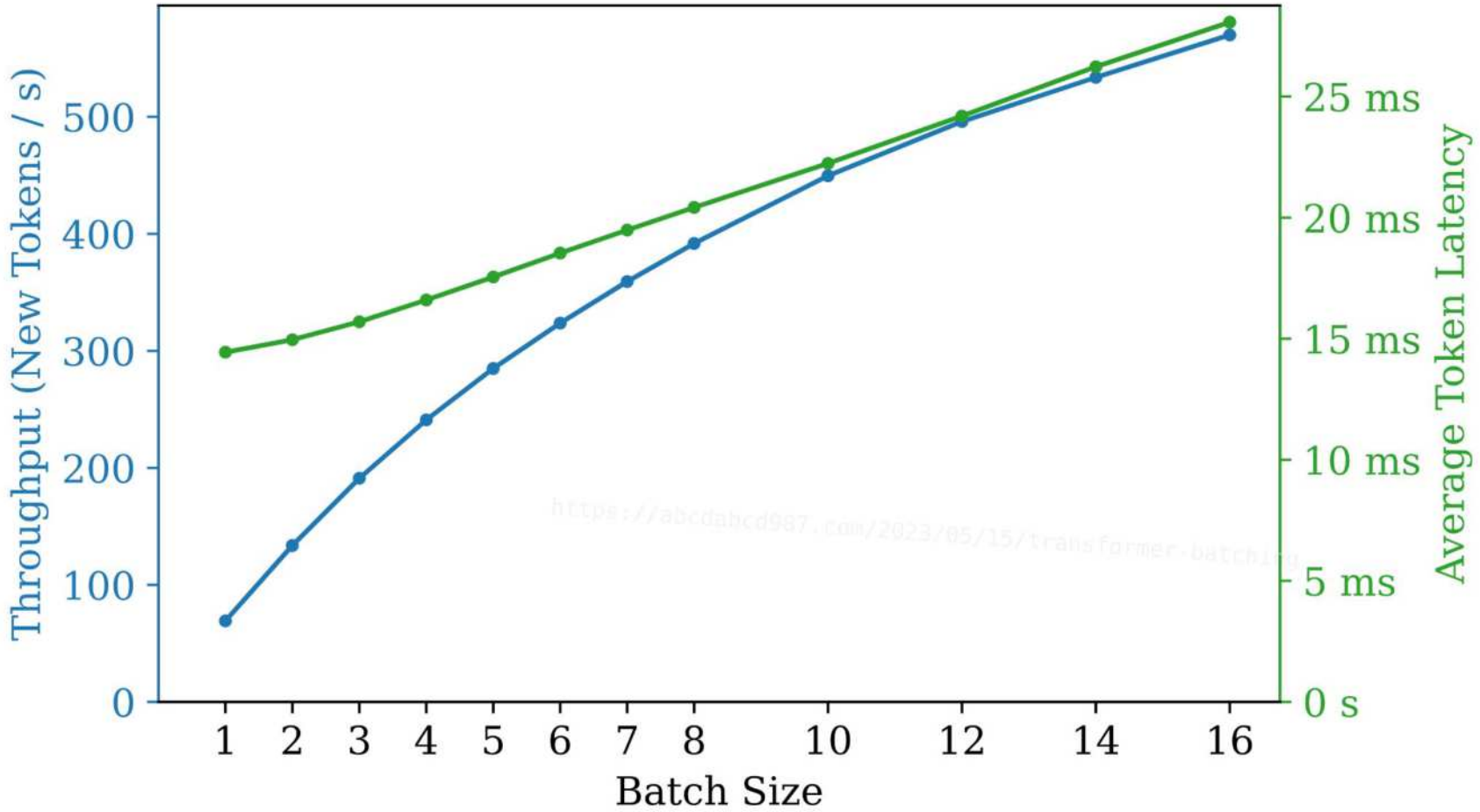
- Switched LLM inference from A100 40GB → H100 80GB
- Used FP8 quantization enabled by newer Tensor Cores:
  - ~2.5× more users handled per GPU
  - Faster text generation (~10%)
- Lower total serving cost despite higher $/GPU-hour
- Achieving good LLM performance and unit economics requires understanding the entire stack

What we will cover today

- GPU
- Memory hierarchy
- CUDA Programming model
- Roofline model
- GPU Profiling
- LLM Inference Basics

Important metrics





https://le.qun.ch/en/blog/2023/05/13/transformer-batching/

Why do we care about GPUs?

Developed as specialized chips in the late 1990s, for graphics processing tasks

Why do we care about GPUs?

Developed as specialized chips in the late 1990s, for graphics processing tasks

GPUs were pivotal in the rise of deep learning (2012) and remain crucial to its ongoing advancements.

Why do we care about GPUs?

Developed as specialized chips in the late 1990s, for graphics processing tasks

GPUs were pivotal in the rise of deep learning (2012) and remain crucial to its ongoing advancements.

Excels in executing many tasks simultaneously, significantly boosting performance in parallel processing scenarios

Why do we care about GPUs?

Expensive and scarce

Monthly estimate

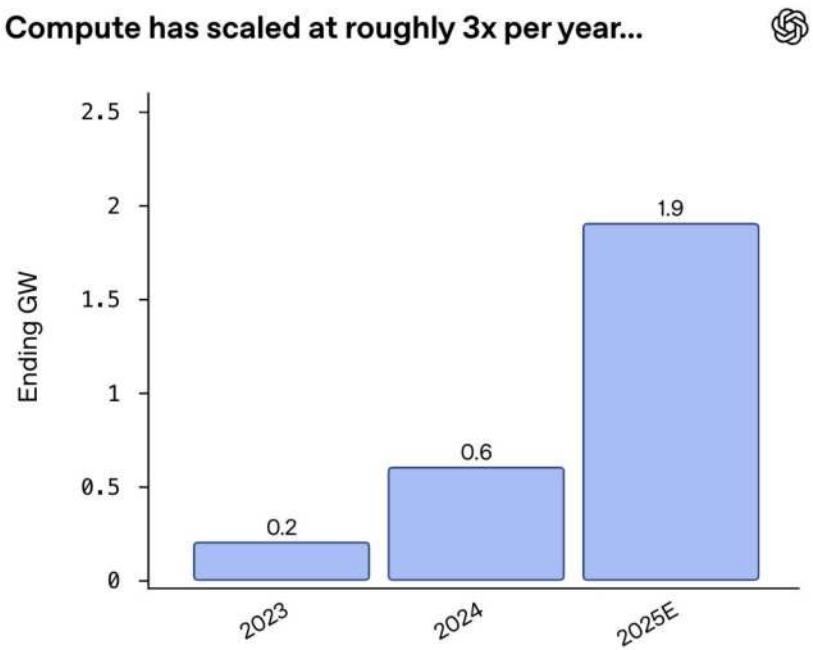
$64,598.70

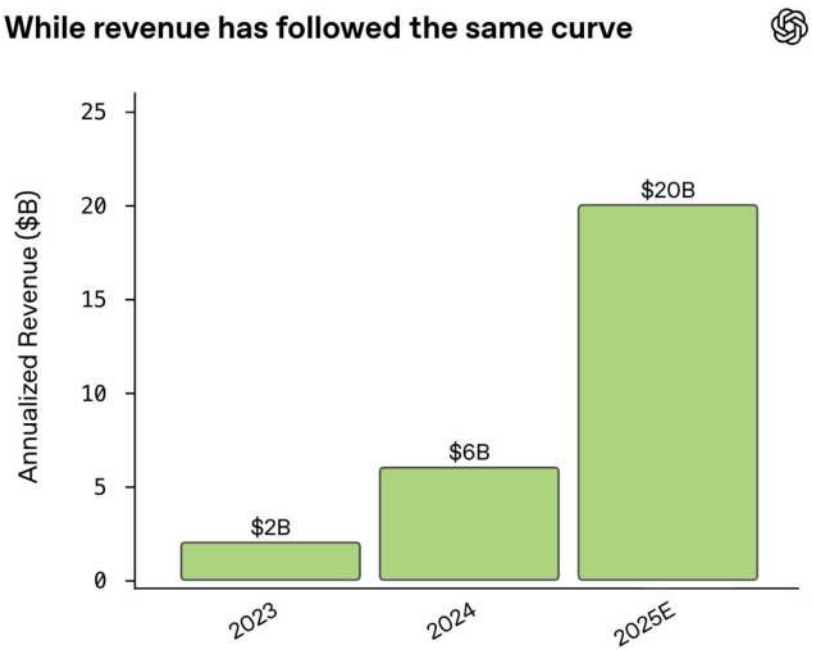
That's about $88.49 hourly

Pay for what you use: no upfront costs and per second billing

Item	Monthly estimate
208 vCPU + 1,872 GB memory	$6,905.84
8 NVIDIA H100 80GB	$57,211.86
6,000 GiB Local SSD disks	$480.00
10 GB balanced persistent disk	$1.00
Logging	Cost varies
Monitoring	Cost varies
Snapshot schedule	Cost varies
Total	$64,598.70

Compute = Revenue



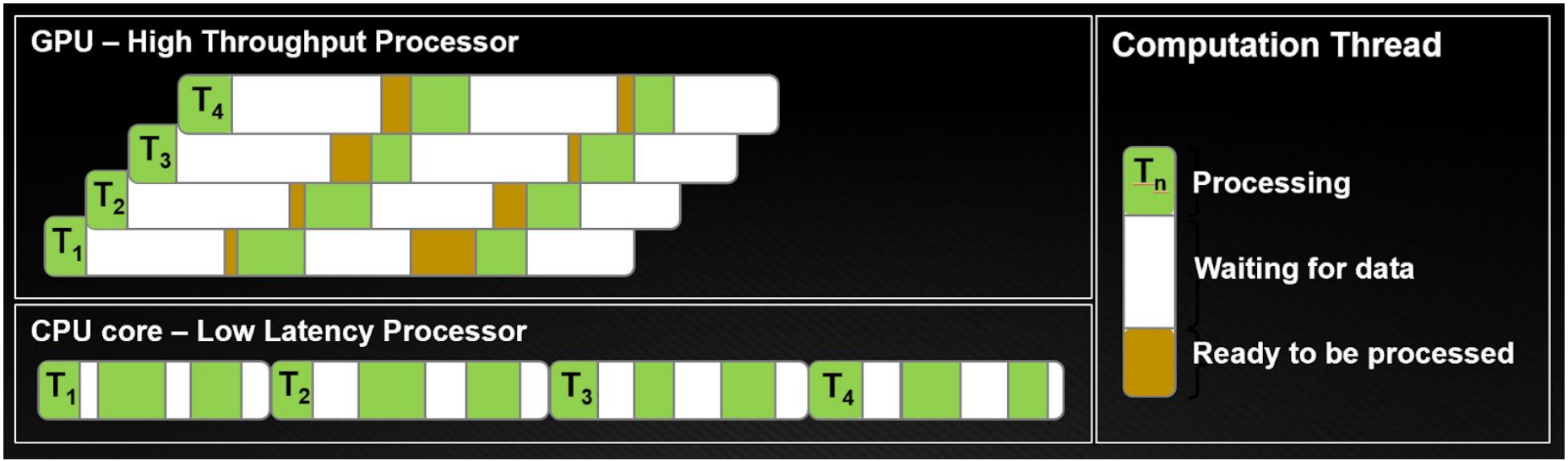


https://openai.com/index/a-business-that-scales-with-the-value-of-intelligence

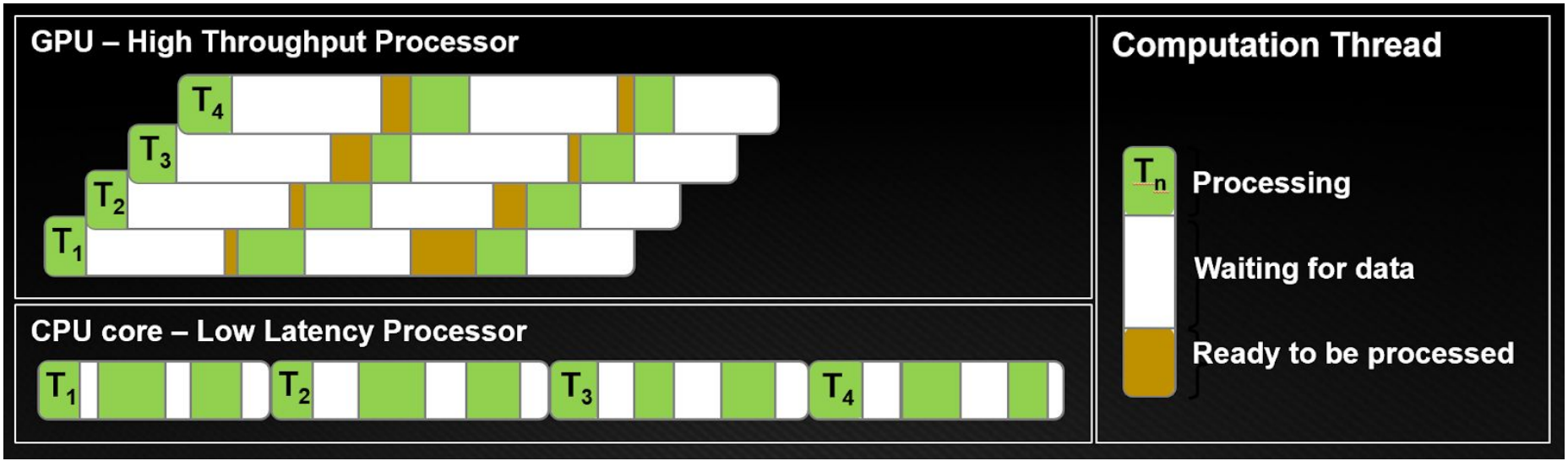
CPU (Central Processing Unit)	**GPU (**Graphics Processing Unit)
Sequential processing (good for complex logic tasks)	Parallel processing (good for repetitive tasks)
Fewer cores, complex cores	Many cores, simpler cores
Random access memory, optimized for low latency	Sequential access to memory, optimized for high bandwidth
Higher clock speeds	Lower clock speeds
Easy to program with general-purpose languages	Programming via specialized languages like CUDA, OpenCL

GPU is specialized for highly parallel computations

Low Latency vs High Throughput



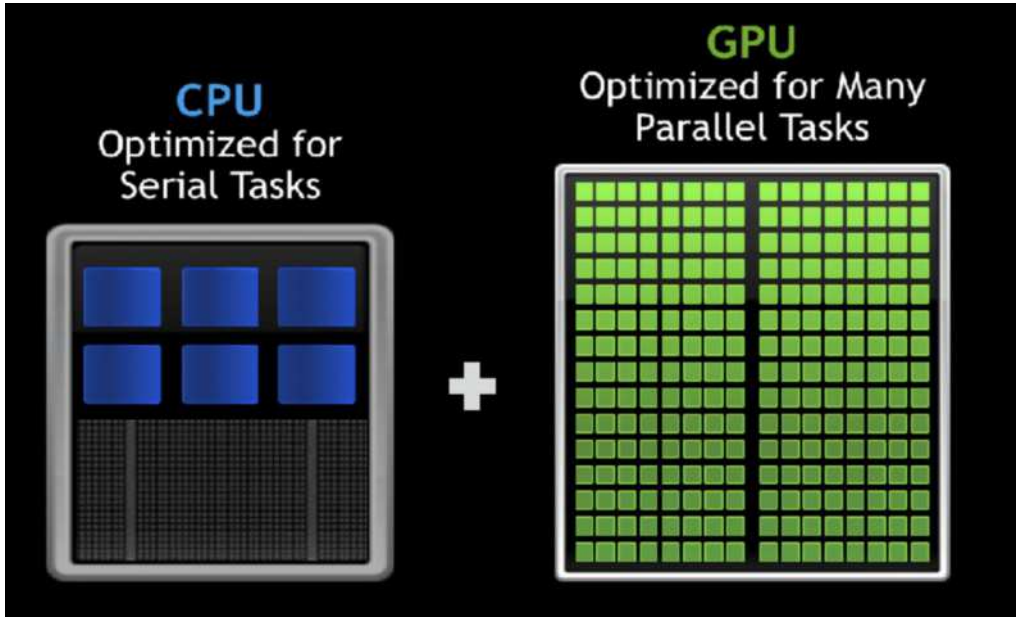
Low Latency vs High Throughput



GPU architecture hides instruction (slower clock cycle) and memory latency with computation.

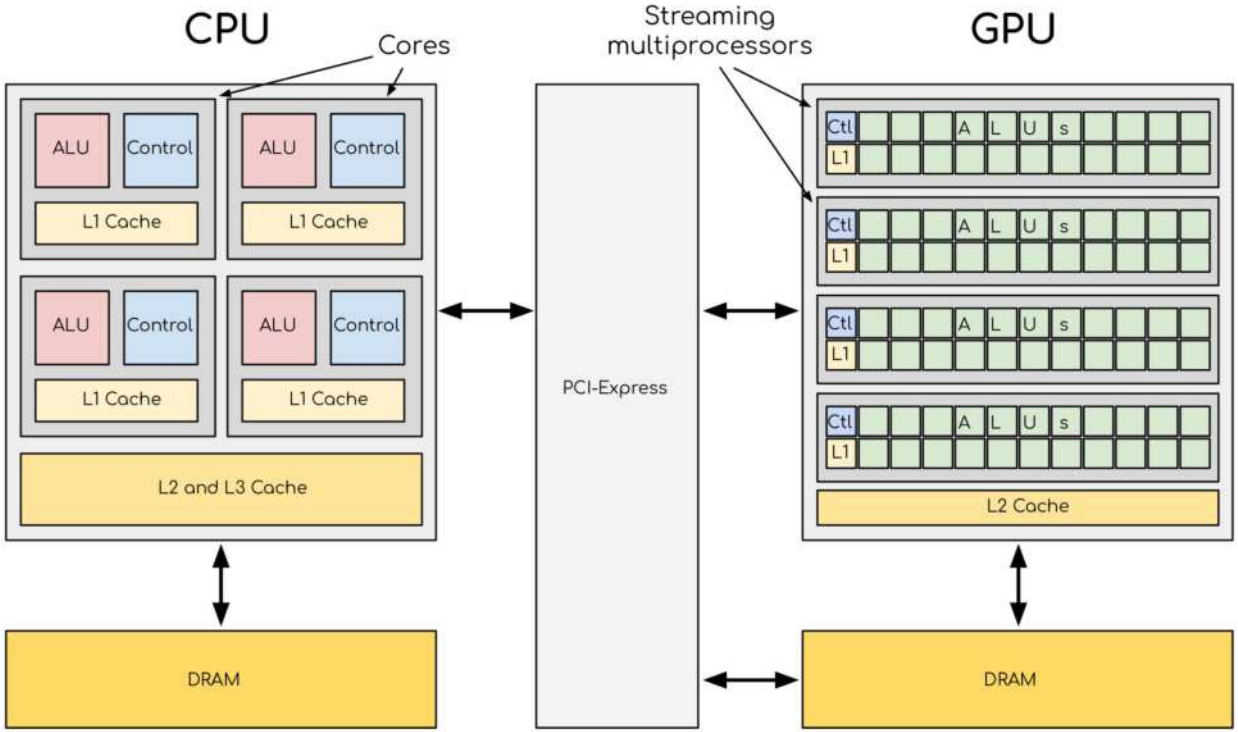
Main takeaways

- CPU → low latency for a single task
- GPU → high throughput for many tasks
- GPUs hide memory latency with massive parallelism
- More concurrent work → higher utilization



GPU Architecture

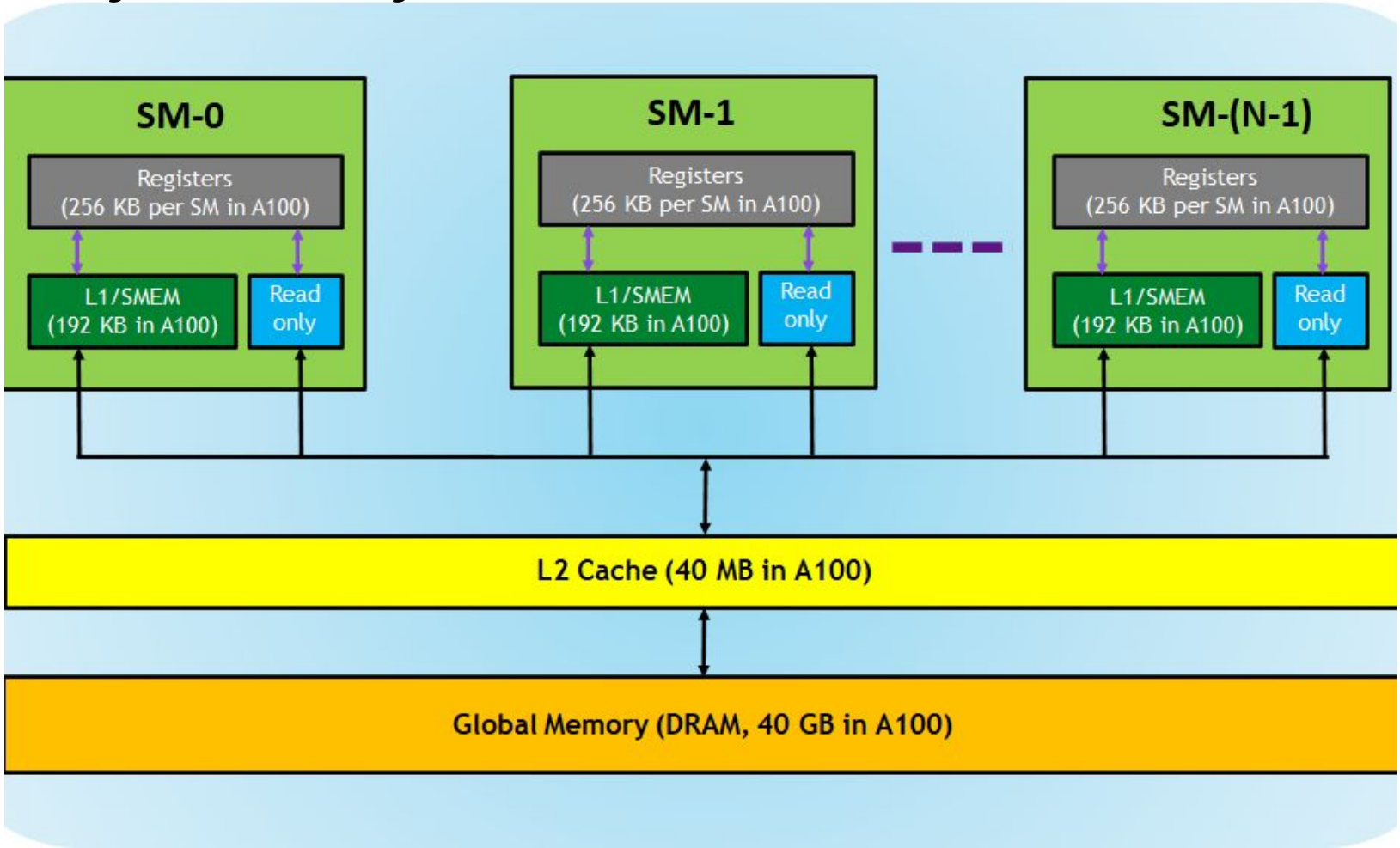
High-level overview



CPU: few cores, large control logic, big caches (L1 / L2 / L3).
GPU: lots of tiny cores, small per-core cache, wide memory bus.

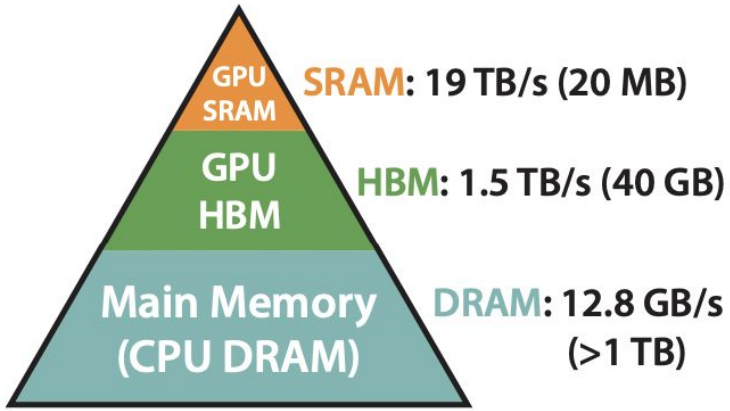
SM Architecture

Memory Hierarchy



Memory Hierarchy

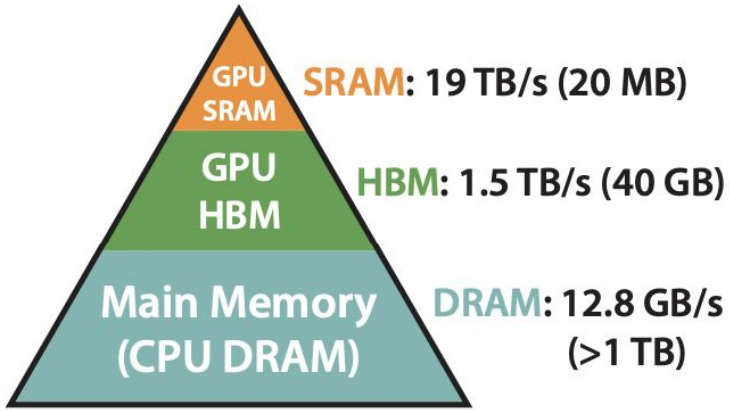
- transfer from host to device is the slowest, minimize them
- data transfer is often a bottleneck



Memory Hierarchy with Bandwidth & Memory Size

Memory Hierarchy

- transfer from host to device is the slowest, minimize them
- data transfer is often a bottleneck

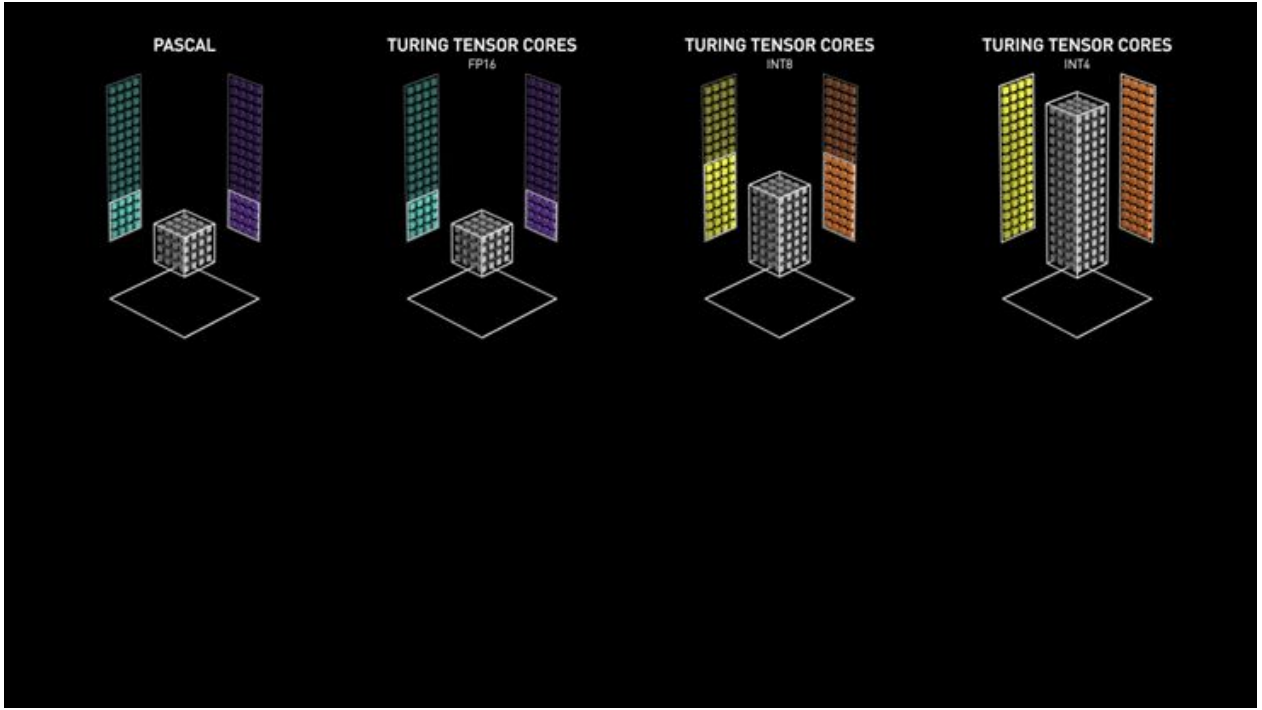


Memory Hierarchy with Bandwidth & Memory Size

Compute speed has outpaced memory speed and many operations in Transformers are bottlenecked by memory accesses

Tensor Cores

Drastically improves the performance of large matrix multiplications



Consumes less power while delivering higher throughput

Tensor Cores

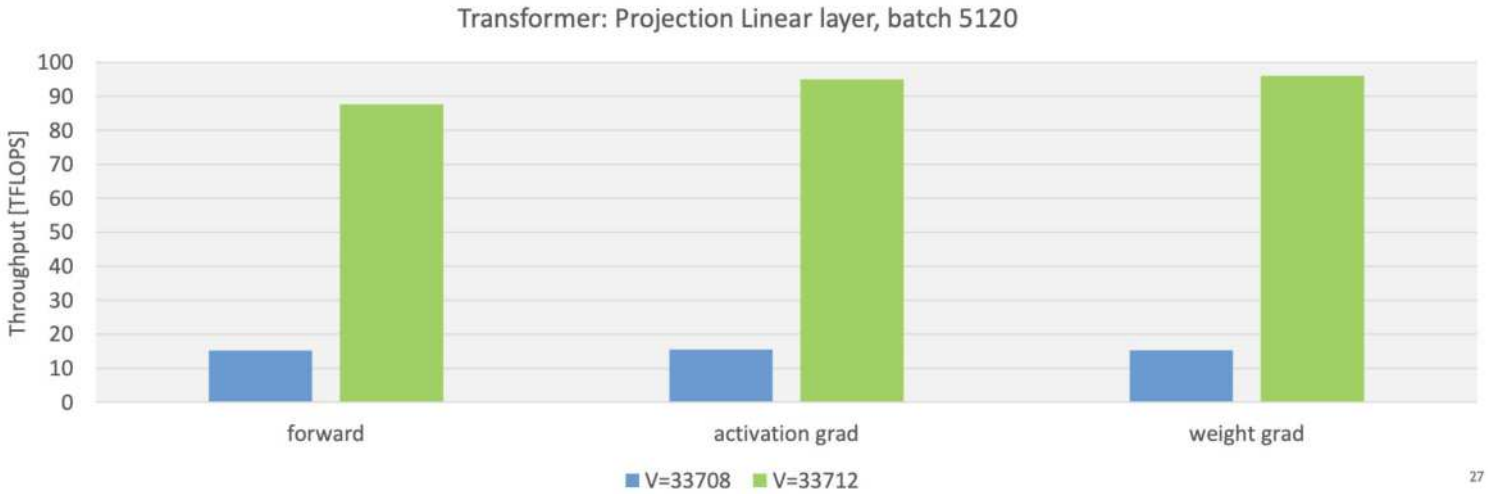
FLOPS (or FLOP/s) - Floating-point Operations Per Second, measurement for computing performance

Technical Specifications	
	H100 SXM
FP64	34 teraFLOPS
FP64 Tensor Core	67 teraFLOPS
FP32	67 teraFLOPS
TF32 Tensor Core*	989 teraFLOPS
BFLOAT16 Tensor Core*	1,979 teraFLOPS
FP16 Tensor Core*	1,979 teraFLOPS
FP8 Tensor Core*	3,958 teraFLOPS
INT8 Tensor Core*	3,958 TOPS
GPU Memory	80GB
GPU Memory Bandwidth	3.35TB/s

https://resources.nvidia.com/en-us-hopper-architecture/nvidia-tensor-core-gpu-datasheet

Tensor Cores

- Automatically used when supported data types and tensor shapes are used
- Can be verified in profilers by checking kernel names

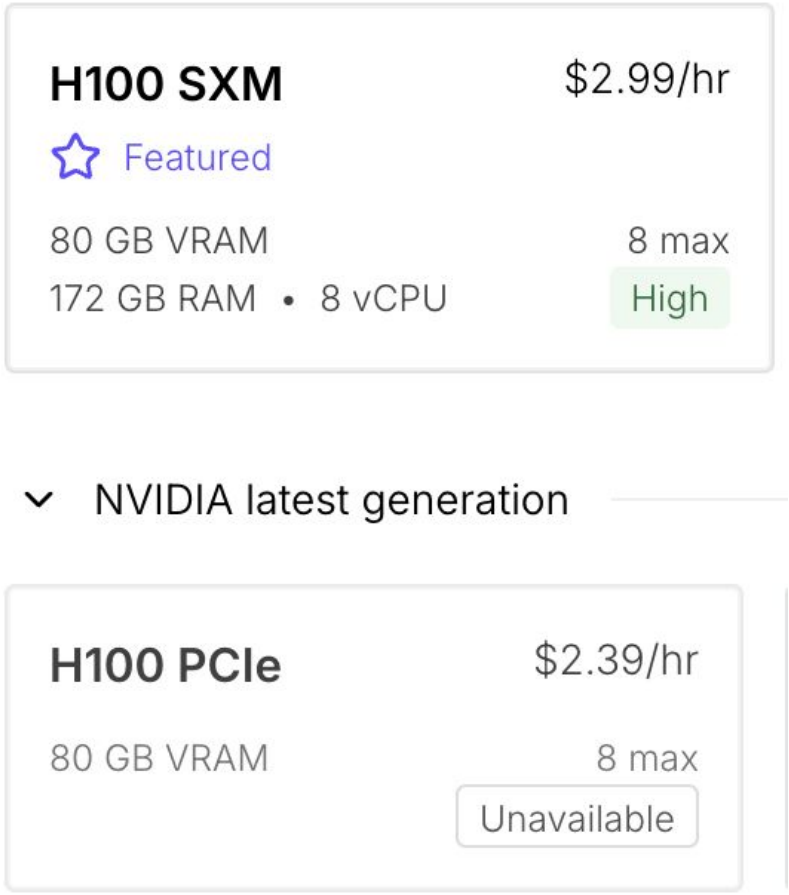


https://developer.download.nvidia.com/video/gputechconf/gtc/2019/presentation/s9926-tensor-core-performance-the-ultimate-guide.pdf

Modern LLMs are too large for a single GPU

Interconnect

- Communication link between multiple GPUs
- Enables fast GPU to GPU data transfer
- PCIe - up to 64 GB/s per direction
- NVLink / NVSwitch - specialized GPU↔GPU fabric; up to 900 GB/s bidirectional per GPU



	GPU0	GPU1	GPU2	GPU3	GPU4	GPU5	GPU6	GPU7	CPU Affinity	NUMA Affinity	GPU NUMA ID
GPU0	X	NV18	NV18	NV18	NV18	NV18	NV18	NV18	0-63	0	N/A
GPU1	NV18	X	NV18	NV18	NV18	NV18	NV18	NV18	0-63	0	N/A
GPU2	NV18	NV18	X	NV18	NV18	NV18	NV18	NV18	0-63	0	N/A
GPU3	NV18	NV18	NV18	X	NV18	NV18	NV18	NV18	0-63	0	N/A
GPU4	NV18	NV18	NV18	NV18	X	NV18	NV18	NV18	64-127	1	N/A
GPU5	NV18	NV18	NV18	NV18	NV18	X	NV18	NV18	64-127	1	N/A
GPU6	NV18	NV18	NV18	NV18	NV18	NV18	X	NV18	64-127	1	N/A
GPU7	NV18	NV18	NV18	NV18	NV18	NV18	NV18	X	64-127	1	N/A

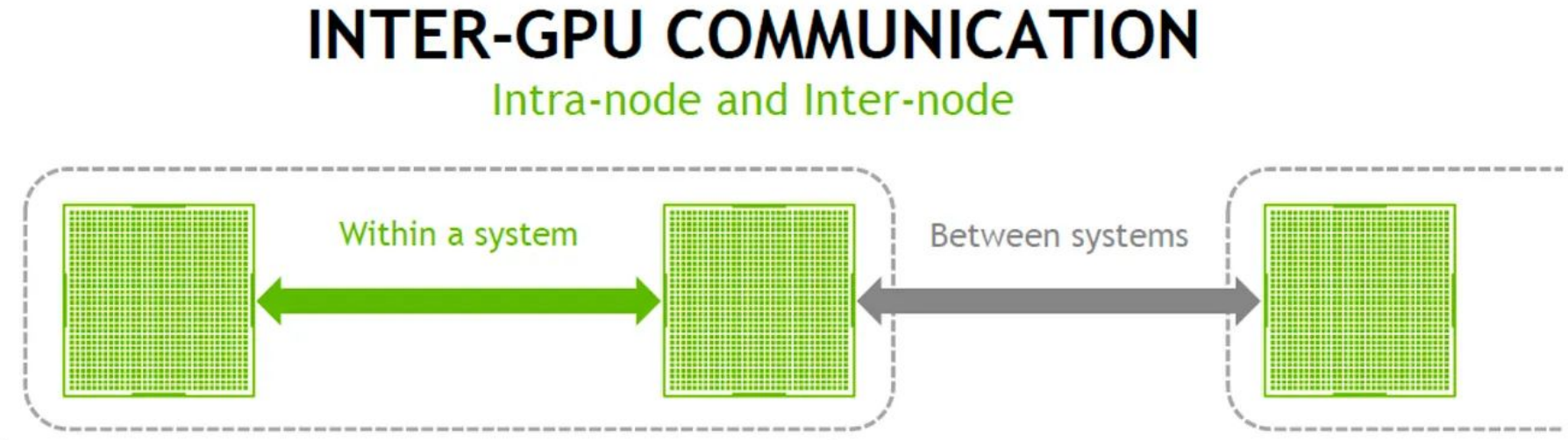
```
$ nvidia-smi topo -m
        GPU0    GPU1    GPU2    GPU3    GPU4    GPU5    GPU6    GPU7    CPU Affinity    NUMA Affinity
GPU0     X      NODE    NODE    NV12    SYS     SYS     SYS     SYS     0-31,64-95      0
GPU1    NODE     X      NV12    PIX     SYS     SYS     SYS     SYS     0-31,64-95      0
GPU2    NODE    NV12     X      PIX     SYS     SYS     SYS     SYS     0-31,64-95      0
GPU3    NV12    PIX     PIX      X      SYS     SYS     SYS     SYS     0-31,64-95      0
GPU4    SYS     SYS     SYS     SYS      X      NV12    NODE    NODE    32-63,96-127    1
GPU5    SYS     SYS     SYS     SYS     NV12     X      NODE    NODE    32-63,96-127    1
GPU6    SYS     SYS     SYS     SYS     NODE    NODE     X      NV12    32-63,96-127    1
GPU7    SYS     SYS     SYS     SYS     NODE    NODE    NV12     X      32-63,96-127    1

Legend:

  X    = Self
  SYS  = Connection traversing PCIe as well as the SMP interconnect between NUMA nodes (e.g., QPI/UPI)
  NODE = Connection traversing PCIe as well as the interconnect between PCIe Host Bridges within a NUMA node
  PHB  = Connection traversing PCIe as well as a PCIe Host Bridge (typically the CPU)
  PXB  = Connection traversing multiple PCIe bridges (without traversing the PCIe Host Bridge)
  PIX  = Connection traversing at most a single PCIe bridge
  NV#  = Connection traversing a bonded set of # NVLinks
```

Scale-up vs Scale-out

- InfiniBand / RoCE / Ethernet
- Slower than NVLink
- Communication becomes expensive



Communication Speed Hierarchy

1 HBM MEMORY

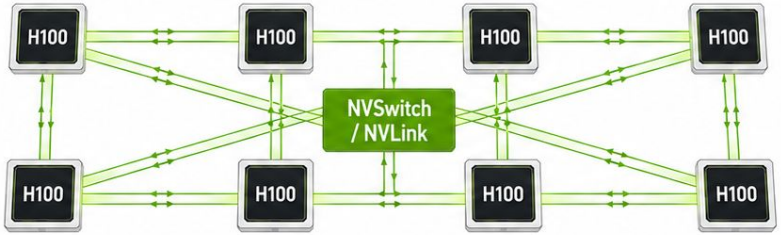
HBM ~3.35 TB/s

Fastest path

2 NVLINK / NVSWITCH

~900 GB/s

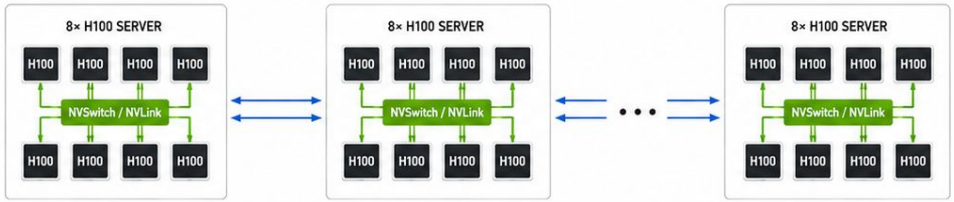
Within one 8×H100 node



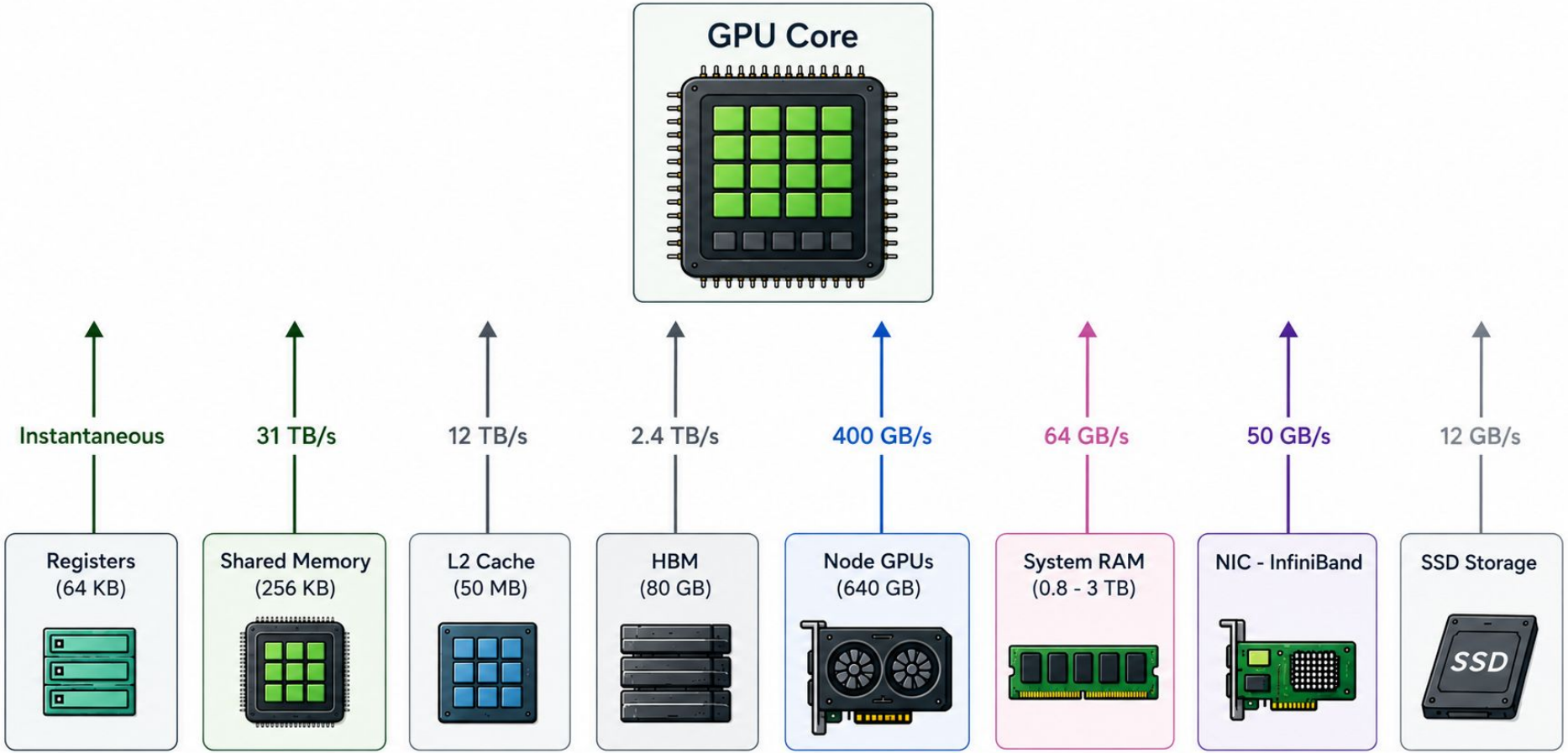
3 INFINIBAND

~50 GB/s per NIC

Between nodes
(per network interface)



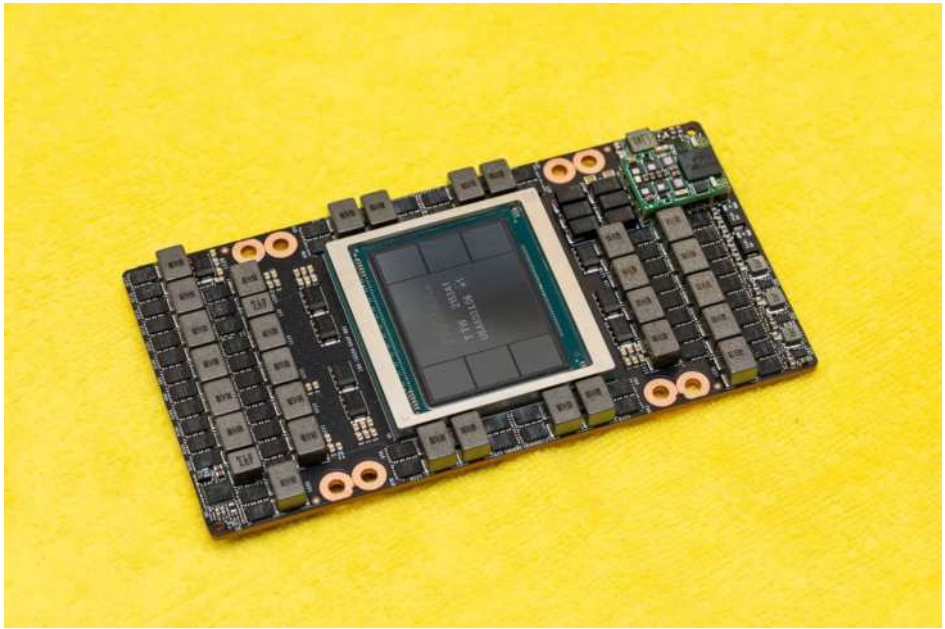
FASTEST — FAST — SLOWER


GPU Core
Instantaneous
31 TB/s
12 TB/s
2.4 TB/s
400 GB/s
64 GB/s
50 GB/s
12 GB/s
Registers
(64 KB)
Shared Memory
(256 KB)
L2 Cache
(50 MB)
HBM
(80 GB)
Node GPUs
(640 GB)
System RAM
(0.8 - 3 TB)
NIC - InfiniBand
SSD Storage
SSD

Takeaways

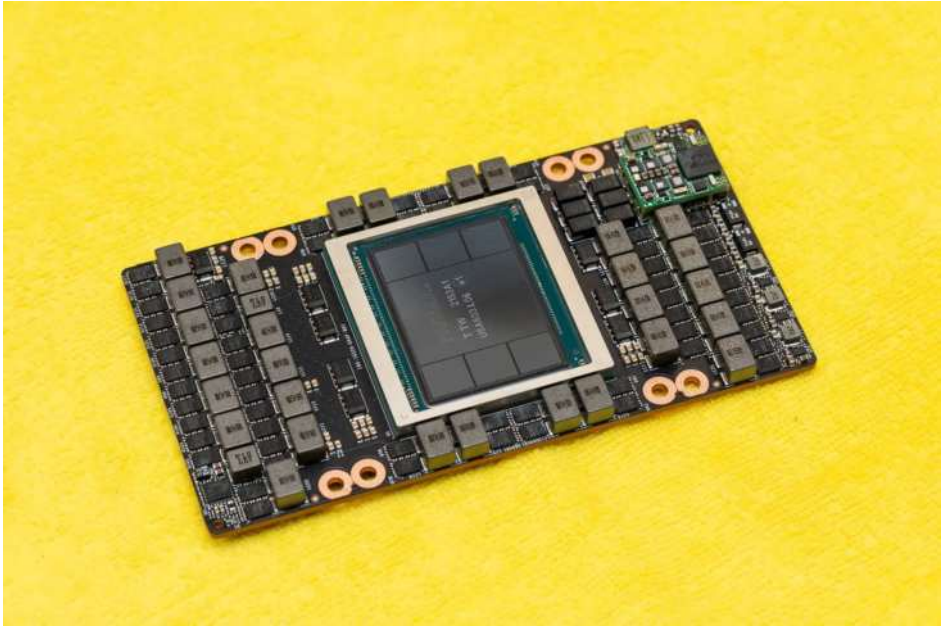
- GPU performance is often limited by communication, not just compute
- Communication bandwidth drops dramatically across the hierarchy:
  - HBM → NVLink/NVSwitch → InfiniBand
- Efficient distributed inference/training requires minimizing cross-node transfers

Important GPU characteristics



Important GPU characteristics

- Compute TFLOPS
- Memory capacity - HBM size
- Memory bandwidth - HBM GB/s
- Interconnect
- Price
- Availability



Example

```python
import time
import torch

dev = torch.device("cuda")
N, ITERS, WARMUP = 8192, 20, 5
FLOPS = 2 * N ** 3  # multiply-add = 2 FLOPs


def bench(run, ref):
    # Warmup
    for _ in range(WARMUP):
        run()
    torch.cuda.synchronize()

    # Time
    t0 = time.perf_counter()
    for _ in range(ITERS):
        out = run()
    torch.cuda.synchronize()
    ms = (time.perf_counter() - t0) * 1e3 / ITERS
    tflops = FLOPS / (ms * 1e-3) / 1e12

    # Accuracy vs fp64 ref
    max_abs_err = (out.double() - ref).abs().max().item()
    rel = max_abs_err / ref.abs().max().item()
    return ms, tflops, rel
```

```python
SPEC = {"FP32 (no TC)": 67, "TF32": 989, "FP16": 1979, "BF16": 1979, "FP8 e4m3": 3958}

results = []
def row(name, fn):
    results.append((name, *bench(fn, ref)))


print(f"GPU: {torch.cuda.get_device_name(0)}, torch {torch.__version__}")
print(f"Matrix: {N}×{N}  ({FLOPS/1e12:.2f} TFLOP/matmul, avg of {ITERS} iters)\n")

torch.manual_seed(0)
a = torch.randn(N, N, device=dev)
b = torch.randn(N, N, device=dev)
ref = a.double() @ b.double()  # FP64 reference for accuracy comparison

# --- FP32 (no tensor cores) ---------------------------------------------
row("FP32 (no TC)", lambda: a @ b)

# --- TF32 ---------------------------------------------------------------
# TF32 is FP32-in / FP32-out but lets the tensor cores round each operand
# to a 19-bit format internally.
torch.backends.cuda.matmul.allow_tf32 = True
row("TF32", lambda: a @ b)
```

```
# --- FP16 ---------------------------------------------------------------
a16, b16 = a.half(), b.half()
row("FP16", lambda: a16 @ b16)

# --- BF16 ---------------------------------------------------------------
abf, bbf = a.bfloat16(), b.bfloat16()
row("BF16", lambda: abf @ bbf)

# --- FP8 e4m3 -----------------------------------------------------------
# Hopper's FP8 MMA requires the right operand in column-major memory.
a8 = a.to(torch.float8_e4m3fn)
b8 = b.to(torch.float8_e4m3fn).t().contiguous().t()
s1 = torch.tensor(1.0, device=dev)
row("FP8 e4m3", lambda: torch._scaled_mm(a8, b8, scale_a=s1, scale_b=s1,
                                         out_dtype=torch.float32))

# --- Results ------------------------------------------------------------
print(f"{'dtype':<14}{'TFLOP/s':>10}{'spec':>9}{'% peak':>9}")
for name, _, tflops, _ in results:
    spec = SPEC[name]
    print(f"{name:<14}{tflops:10.1f}{spec:9}{100*tflops/spec:8.1f}%")
```

Tensor Cores: Real Speedup

GPU: NVIDIA H100 80GB HBM3, torch 2.11.0+cu128
Matrix: 8192×8192 (1.10 TFLOP/matmul, avg of 20 iters)

dtype	ms	TFLOP/s	spec	%peak	rel err	speedup
FP32 (no TC)	22.13	49.7	67	74.1%	4.28e-06	1.0x
TF32	2.66	413.3	989	41.8%	2.98e-04	8.3x
FP16	1.45	759.5	1979	38.4%	4.36e-04	15.3x
BF16	1.40	787.4	1979	39.8%	3.51e-03	15.8x
FP8 e4m3	0.80	1370.8	3958	34.6%	3.81e-02	27.6x

Specs Can Be Misleading

- sparsity
- memory traffic
- thermal
- lower precision is harder to feed fast enough
- fixed overheads matter more

```
GPU: NVIDIA H100 80GB HBM3, torch 2.11.0+cu128
Matrix: 8192×8192  (1.10 TFLOP/matmul, avg of 20 iters)

dtype                ms    TFLOP/s    spec    %peak     rel err  speedup
FP32 (no TC)      22.13       49.7      67   74.1%    4.28e-06     1.0x
TF32               2.66      413.3     989   41.8%    2.98e-04     8.3x
FP16               1.45      759.5    1979   38.4%    4.36e-04    15.3x
BF16               1.40      787.4    1979   39.8%    3.51e-03    15.8x
FP8 e4m3           0.80     1370.8    3958   34.6%    3.81e-02    27.6x
```

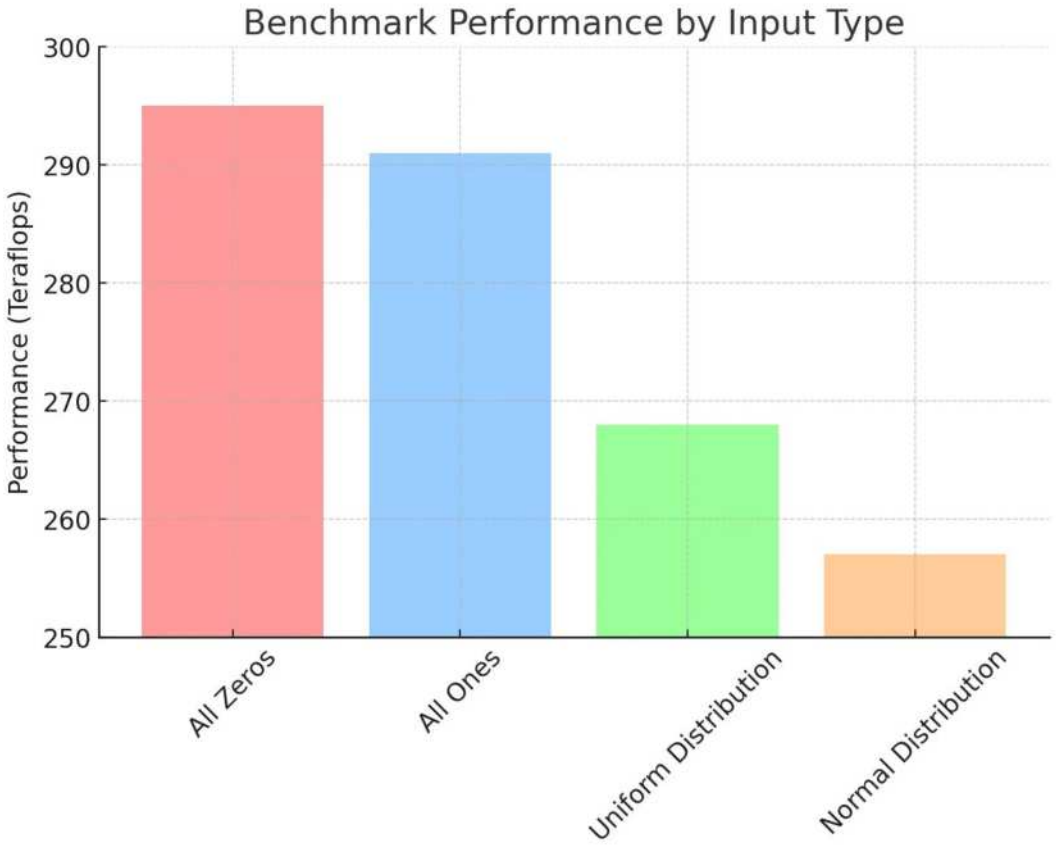
Importance of Shapes

```
GPU: NVIDIA H100 80GB HBM3, torch 2.11.0+cu128
Matrix: 8191×8191  (1.10 TFLOP/matmul, avg of 20 iters)

dtype                ms   TFLOP/s   spec   %peak    rel err  speedup
FP32 (no TC)      22.44      49.0     67   73.1%   4.57e-06     1.0x
TF32               7.79     141.1    495   28.5%   3.06e-04     2.9x
FP16               7.12     154.4    989   15.6%   5.60e-04     3.2x
BF16               7.08     155.2    989   15.7%   5.31e-03     3.2x
```

TFLOP/s depends on the input as well

- GPUs have a power limit to prevent overheating
- Random inputs cause more transistor switching
  → higher dynamic power
  → clock throttling
- Lower sustained clocks
  → lower TFLOP/s



https://www.thonking.ai/p/strangely-matrix-multiplications

Example Summary

- Specs ≠ real performance
- Shapes matter
- Kernel selection matters
- Measure, don’t assume

GPU

CUDA programming model

CUDA Programming Model

- Host - CPU and its memory
- Device - GPU and its memory
- Kernel - functions, executed N times in parallel by N different CUDA threads

```
// CUDA kernel for adding two arrays
__global__ void addKernel(int *c, const int *a, const int *b)
{
    int i = threadIdx.x; // Get the thread index
    c[i] = a[i] + b[i];  // Simple addition
}
```

Same instructions, different data

```
// CUDA kernel for adding two arrays
__global__ void addKernel(int *c, const int *a, const int *b)
{
    int i = threadIdx.x; // Get the thread index
    c[i] = a[i] + b[i];  // Simple addition
}
```

We launch a **kernel** from the **host**, executed in parallel on the **device**

```
// Allocate memory on host: a, b, c
// Allocate memory on device with cudaMalloc: dev_a, dev_b, dev_c
// Copy input arrays to the device
cudaMemcpy(dev_a, a, arraySize * sizeof(int), cudaMemcpyHostToDevice);
cudaMemcpy(dev_b, b, arraySize * sizeof(int), cudaMemcpyHostToDevice);

// Launch the kernel
addKernel<<<1, arraySize>>>(dev_c, dev_a, dev_b);

// Copy the result back to the host
cudaMemcpy(c, dev_c, arraySize * sizeof(int), cudaMemcpyDeviceToHost);

// Deallocate
```

Blocking and non-blocking calls

Blocking (synchronous)

- Host **waits** until operation finishes
- Ensures correct ordering
- Examples:
  - cudaMemcpy
  - cudaDeviceSynchronize
  - tensor.item() → synchronous D2H copy

Non-Blocking (asynchronous)

- Host continues execution immediately
- CPU and GPU work in parallel
- Examples:
  - kernel launch
  - cudaMemcpyAsync
- Helps hide latency and improve utilization (e.g. processing & preparing next inputs)

```
import torch

N = 8192
a = torch.rand(N, N, device="cuda")
```

✓ 0.0s

```
%%time
b = a @ a @ a @ a
```

✓ 0.0s

```
CPU times: user 785 µs, sys: 114 µs, total: 899 µs
Wall time: 431 µs
```

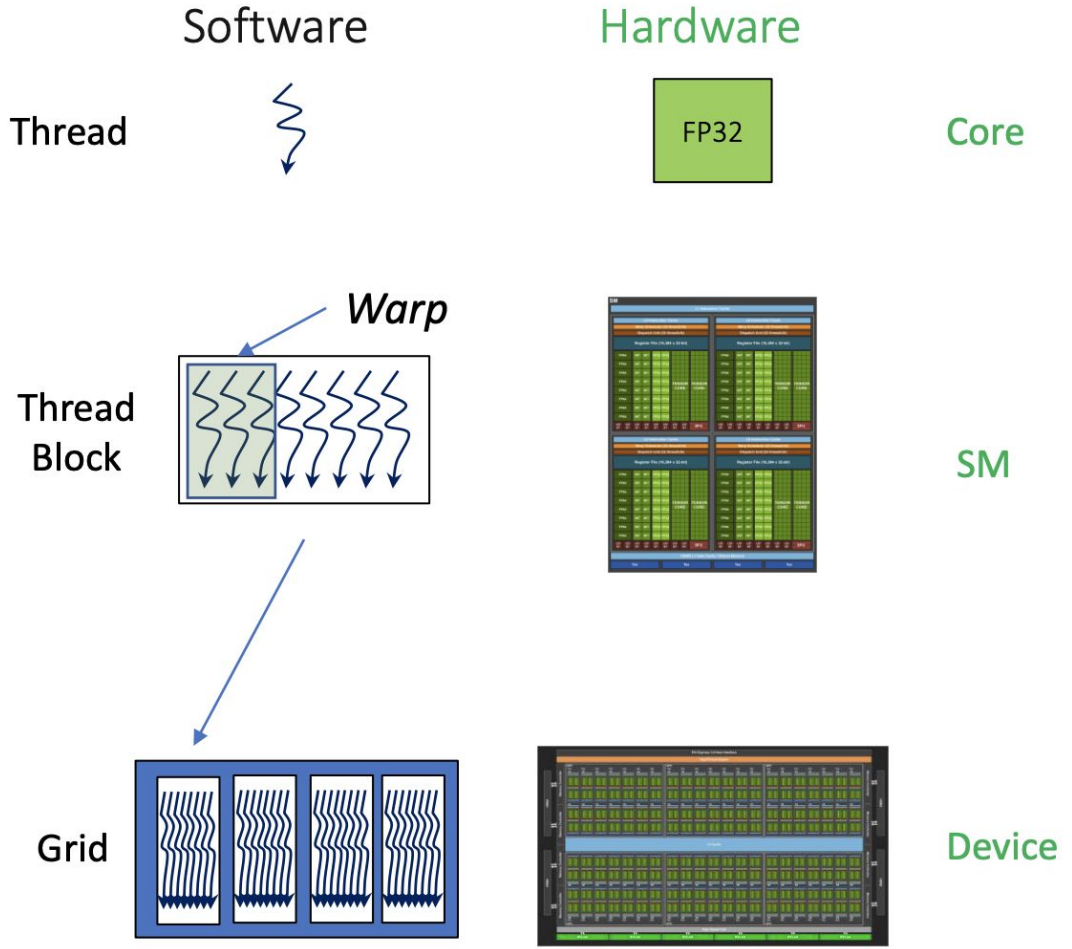
```
%%time
b = a @ a @ a @ a
torch.cuda.synchronize()
```

✓ 0.0s

```
CPU times: user 63.7 ms, sys: 934 µs, total: 64.6 ms
Wall time: 63.9 ms
```

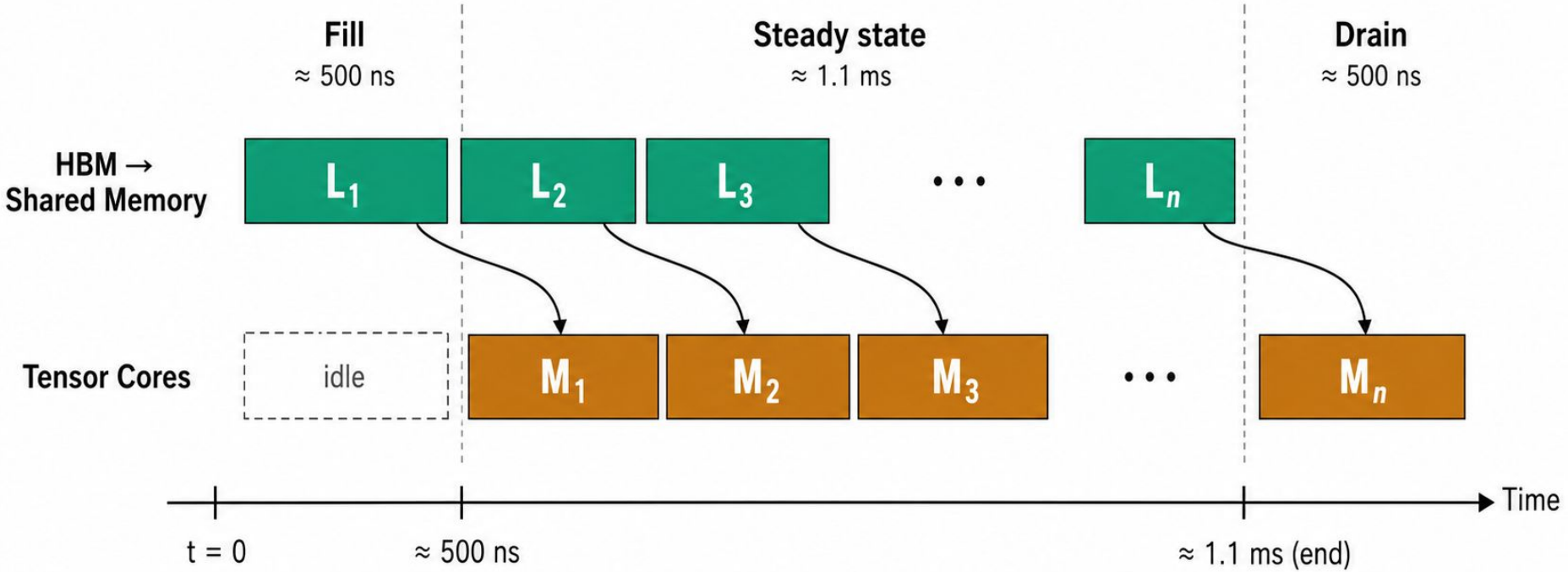
CUDA Programming Model

- **thread** - the smallest unit of processing that can be scheduled by the GPU
- **warp** is a specific number of threads that are executed simultaneously by a GPU (typically = 32)
- threads in a warp start and end at the same time (which can lead to idle threads)



GEMM pipeline inside one SM — H100, BF16 tensor cores

8192 × 8192 matrix multiply, ~1.1 ms total



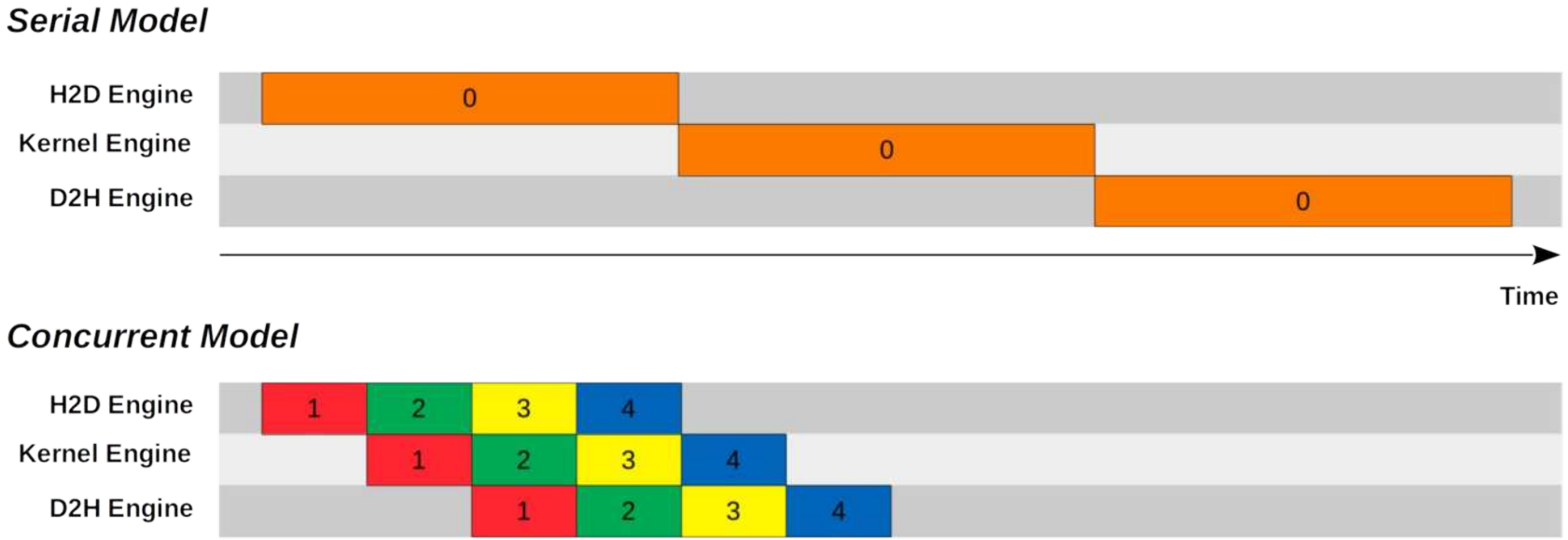
Memory transfer is hidden underneath compute.

Fill + drain ≈ 0.1% of total runtime — the kernel is compute-bound.

Streams - even better concurrent mechanism

Represents a sequence of operations that are executed on the GPU

Steams can be executed concurrently



```python
compute_N = 8192
copy_N = 12288

A = torch.rand(compute_N, compute_N, device="cuda")

# pin_memory=True makes CPU memory page-locked,
# so CUDA can copy from it asynchronously.
CPU_INPUT = torch.rand(copy_N, copy_N, pin_memory=True)

compute_stream = torch.cuda.Stream()
copy_stream = torch.cuda.Stream()


def sequential_execution():
    compute_result = A @ A
    copy_result = CPU_INPUT.to("cuda", non_blocking=True)
    return compute_result, copy_result


def stream_execution():
    with torch.cuda.stream(compute_stream):
        compute_result = A @ A
    with torch.cuda.stream(copy_stream):
        copy_result = CPU_INPUT.to("cuda", non_blocking=True)
    return compute_result, copy_result
```

✓ 0.7s

```python
compute_N = 8192
copy_N = 12288

A = torch.rand(compute_N, compute_N, device="cuda")

# pin_memory=True makes CPU memory page-locked,
# so CUDA can copy from it asynchronously.
CPU_INPUT = torch.rand(copy_N, copy_N, pin_memory=True)

compute_stream = torch.cuda.Stream()
copy_stream = torch.cuda.Stream()


def sequential_execution():
    compute_result = A @ A
    copy_result = CPU_INPUT.to("cuda", non_blocking=Tr
    return compute_result, copy_result


def stream_execution():
    with torch.cuda.stream(compute_stream):
        compute_result = A @ A
    with torch.cuda.stream(copy_stream):
        copy_result = CPU_INPUT.to("cuda", non_blocking=True)
    return compute_result, copy_result
```

✓ 0.7s

```
%%timeit
out = sequential_execution()
torch.cuda.synchronize()
```

✓ 2.6s

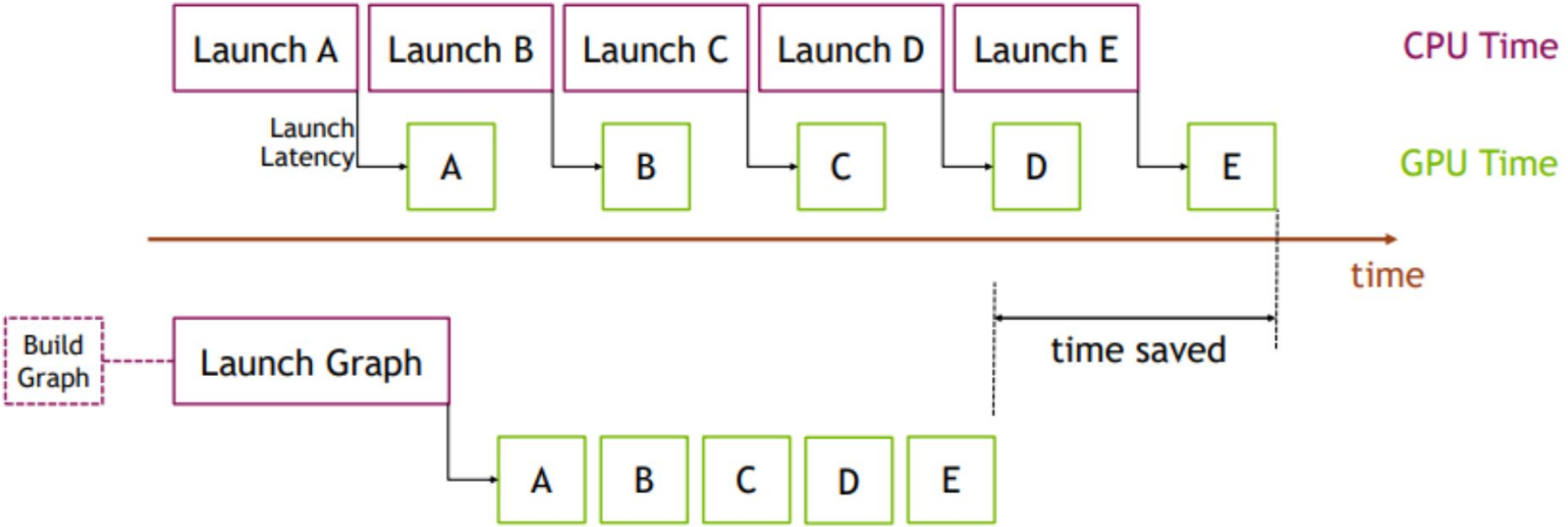
```
32.1 ms ± 13.6 µs per loop (mean ± std. dev. of 7 runs, 10 loops each)
```

```
%%timeit
out = stream_execution()
torch.cuda.synchronize()
```

✓ 1.7s

```
21.3 ms ± 3.88 µs per loop (mean ± std. dev. of 7 runs, 10 loops each)
```

CUDA Graphs


Launch A
Launch B
Launch C
Launch D
Launch E
CPU Time
Launch Latency
A
B
C
D
E
GPU Time
time
Build Graph
Launch Graph
time saved
A
B
C
D
E

```python
graph_N = 512
graph_steps = 100

static_x = torch.rand(graph_N, graph_N, device="cuda")


def tiny_workload(x):
    y = x.clone()
    for _ in range(graph_steps):
        y = y * 1.0001
        y = y + 0.0001
    return y


graph = torch.cuda.CUDAGraph()
with torch.cuda.graph(graph):
    graph_output = tiny_workload(static_x)
torch.cuda.synchronize()
```

✓ 0.0s

```python
graph_N = 512
graph_steps = 100

static_x = torch.rand(graph_N, graph_N, device="cuda")


def tiny_workload(x):
    y = x.clone()
    for _ in range(graph_steps):
        y = y * 1.0001
        y = y + 0.0001
    return y


graph = torch.cuda.CUDAGraph()
with torch.cuda.graph(graph):
    graph_output = tiny_workload(static_x)
torch.cuda.synchronize()
```

✓ 0.0s

```
%%timeit
out = tiny_workload(static_x)
torch.cuda.synchronize()
```

✓ 9.9s

```
1.22 ms ± 11 µs per loop (mean ± std. dev. of 7 runs, 1,000 loops each)
```

```
%%timeit
graph.replay()
torch.cuda.synchronize()
```

✓ 2.7s

```
339 µs ± 27.6 ns per loop (mean ± std. dev. of 7 runs, 1,000 loops each)
```

GPU

Roofline Model

Two Ceilings

- Compute ceiling - how quickly the cores can execute floating-point operations (peak FLOP/s)
- Memory ceiling – how quickly the memory system can feed those cores with data (peak bytes/s)
- For H100:
  - ~989 TFLOP/s
  - 3.35 TB/s

Arithmetic Intensity

1. How much compute is done per byte of memory moved?

$$\text{arithmetic intensity} = \frac{\text{total FLOPs performed}}{\text{total bytes moved from memory}}$$

Arithmetic Intensity

1. How much compute is done per byte of memory moved?

$$\text{arithmetic intensity} = \frac{\text{total FLOPs performed}}{\text{total bytes moved from memory}}$$

2. What performance is achieved?

Arithmetic Intensity

1. How much compute is done per byte of memory moved?

$$\text{arithmetic intensity} = \frac{\text{total FLOPs performed}}{\text{total bytes moved from memory}}$$

2. What performance is achieved?

$$\text{achieved performance} = \frac{\text{total FLOPs performed}}{\text{execution time}}$$

Arithmetic Intensity: Dot Product Example

$$\text{Intensity}(\text{dot product}) = \frac{\text{Total FLOPs}}{\text{Total Bytes}} =$$

Arithmetic Intensity: Dot Product Example

$$\text{Intensity}(\text{dot product}) = \frac{\text{Total FLOPs}}{\text{Total Bytes}} = \frac{N + N - 1}{2N + 2N + 2} = \frac{2N - 1}{4N + 2} \to \frac{1}{2}$$

Low Arithmetic Intensity →

High Arithmetic Intensity →

Low Arithmetic Intensity → Memory Bound

High Arithmetic Intensity → Compute Bound

Matrix Multiplication Example

$$\text{total FLOPs} \approx$$

$$\text{bytes moved} \approx$$

Matrix Multiplication Example

$$\text{total FLOPs} \approx 2N^3$$

$$\text{bytes moved} \approx 3N^2 \times 2 = 6N^2$$

$$\text{arithmetic intensity} = \frac{2N^3}{6N^2} = \frac{N}{3}$$

Where is the bottleneck?

$$\text{achieved performance} = \frac{\text{total FLOPs performed}}{\text{execution time}}$$

Where is the bottleneck?

$$\text{achieved performance} = \frac{\text{total FLOPs performed}}{\text{execution time}}$$

$$= \frac{\text{total FLOPs performed}}{\text{execution time}} \cdot \frac{\text{total bytes moved}}{\text{total bytes moved}}$$

Where is the bottleneck?

$$\text{achieved performance} = \frac{\text{total FLOPs performed}}{\text{execution time}}$$

$$= \frac{\text{total FLOPs performed}}{\text{execution time}} \cdot \frac{\text{total bytes moved}}{\text{total bytes moved}}$$

$$= \text{arithmetic intensity} \cdot \text{memory bandwidth}$$

Where is the bottleneck?

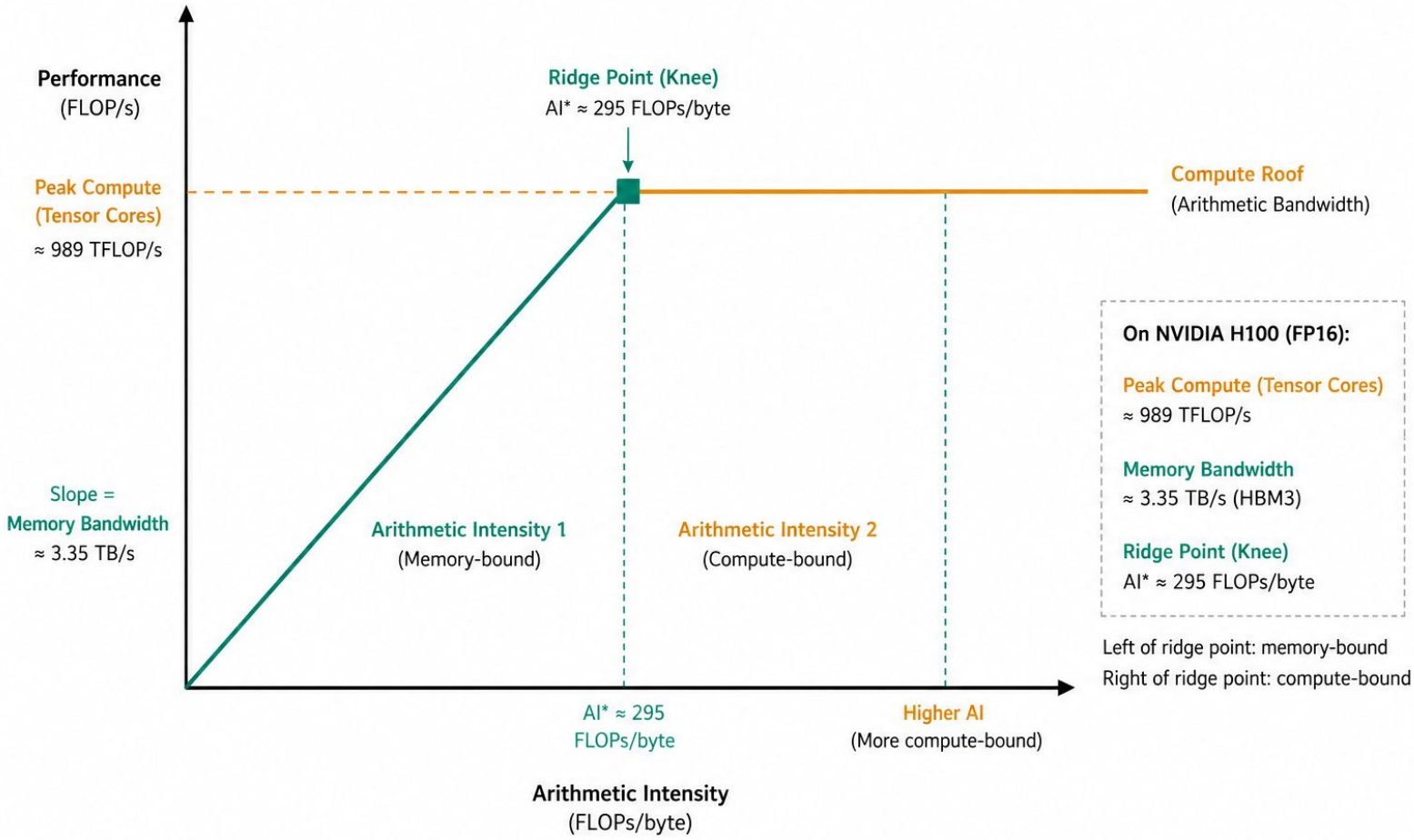
$$\text{achieved performance} = \frac{\text{total FLOPs performed}}{\text{execution time}}$$

$$= \frac{\text{total FLOPs performed}}{\text{execution time}} \cdot \frac{\text{total bytes moved}}{\text{total bytes moved}}$$

$$= \text{arithmetic intensity} \cdot \text{memory bandwidth}$$

$$\leq \min\left(\text{peak compute},\ \text{arithmetic intensity} \times \text{peak memory bandwidth}\right)$$

Roofline Model for NVIDIA H100 (FP16 Tensor Cores)

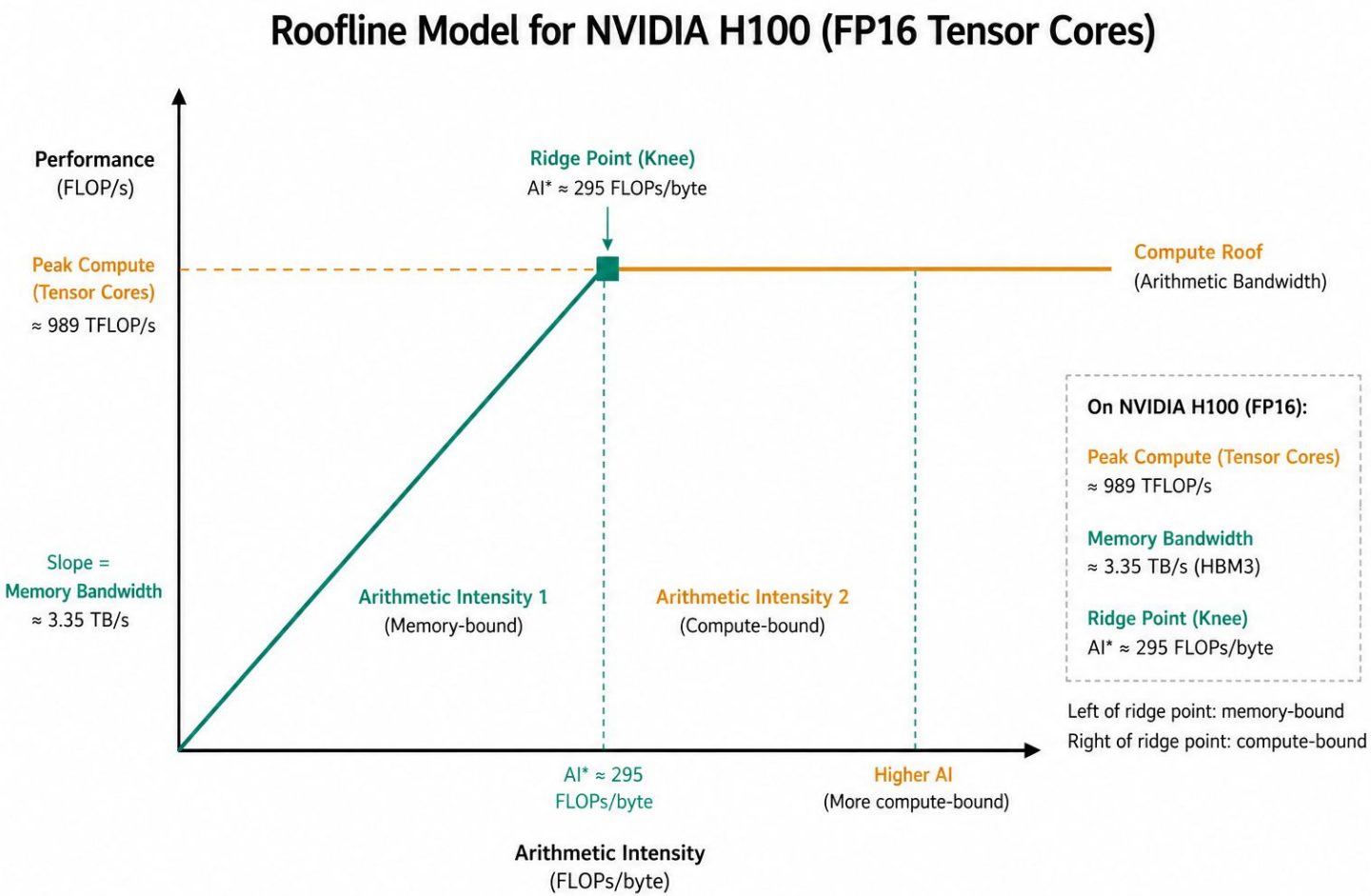


Memory-bound:

- move less data
- reuse loaded data better

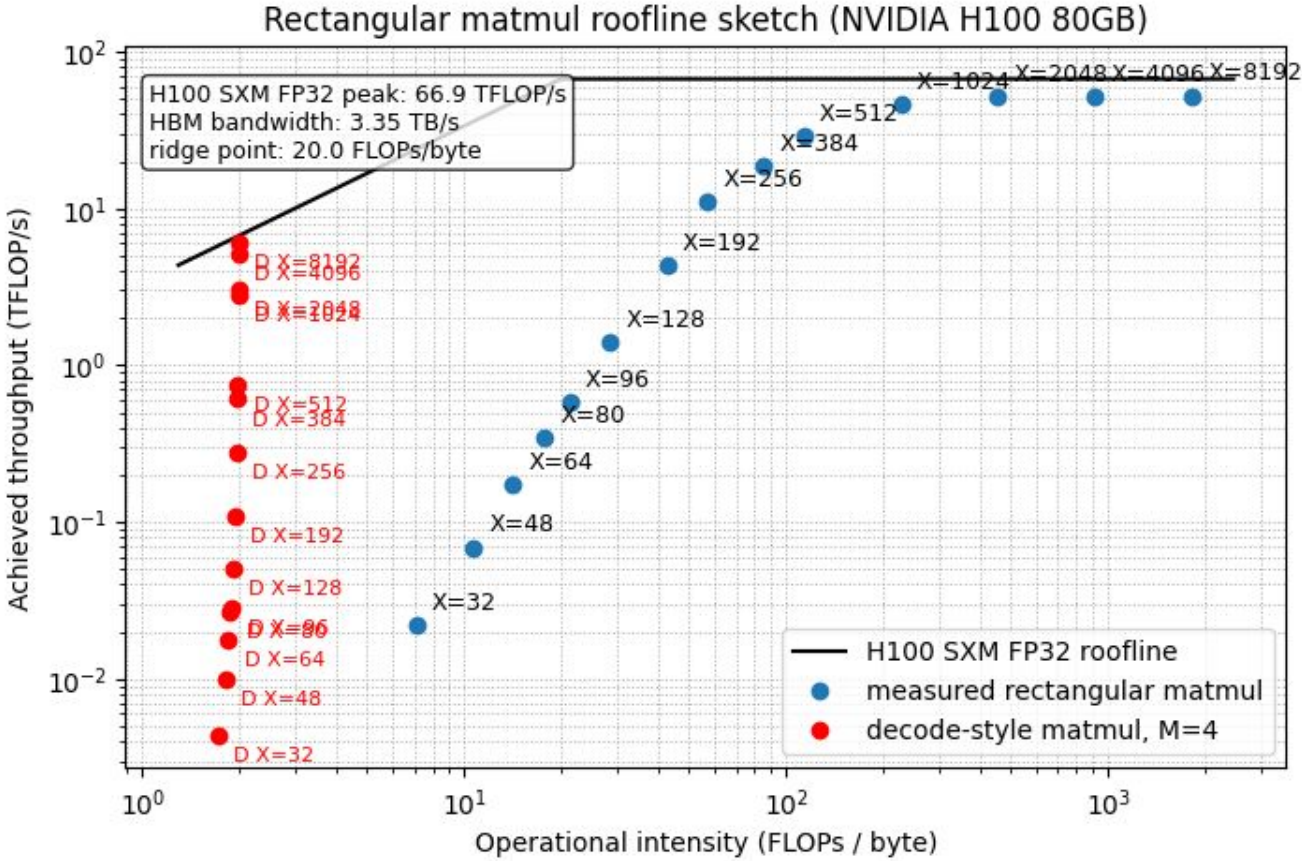
Compute-bound:

- use faster cores
- keep compute units busy



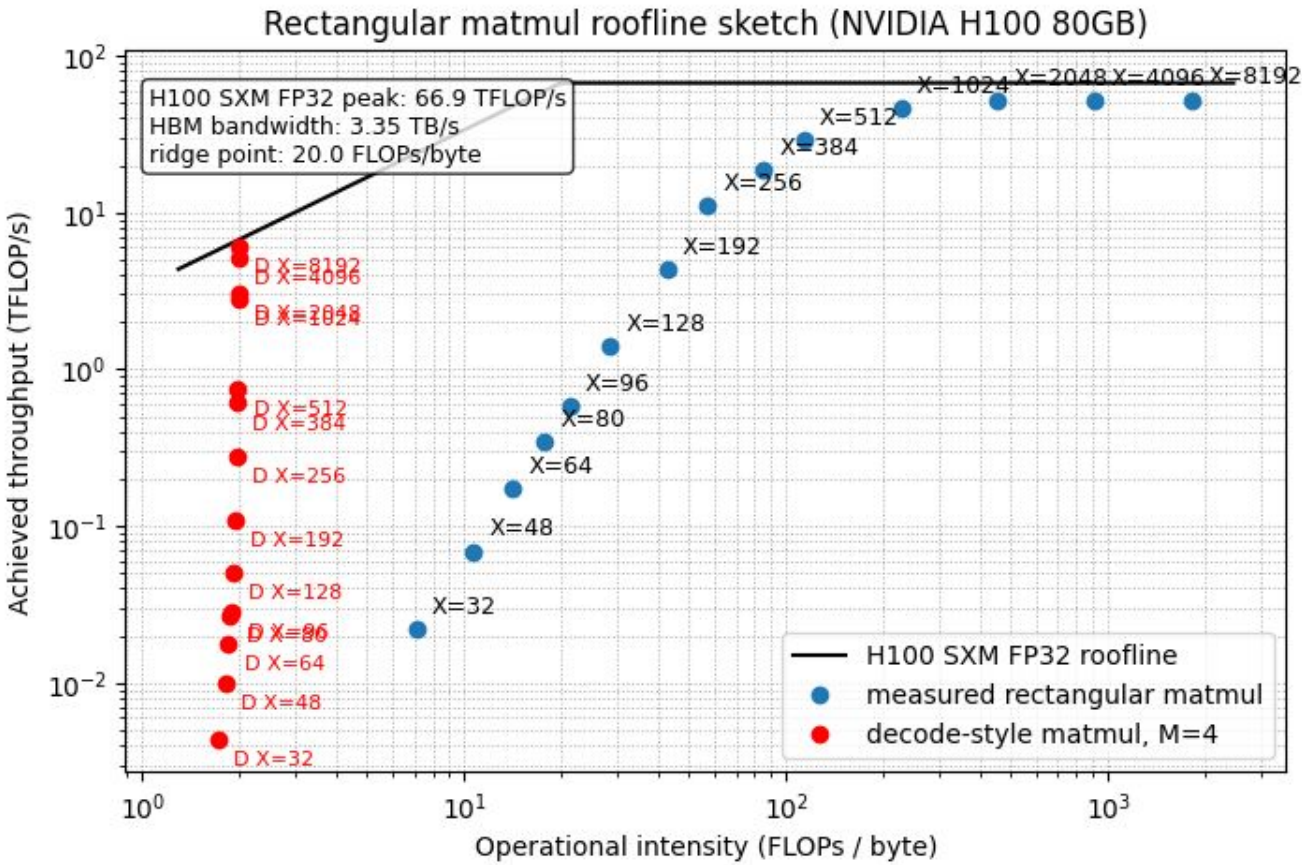
Not reaching the ceiling:

- Roofline assumes ideal peak compute and memory bandwidth.
- Real kernels lose cycles to memory stalls, synchronization, scheduling, and instruction overhead.
- Small or awkward shapes leave Tensor Cores / SMs underutilized.



Getting closer to the ceiling

- Larger matmuls reuse loaded data more times.
- More independent work keeps more SMs busy and hides memory latency.
- Fixed overheads become smaller compared with useful computation.



Takeaways

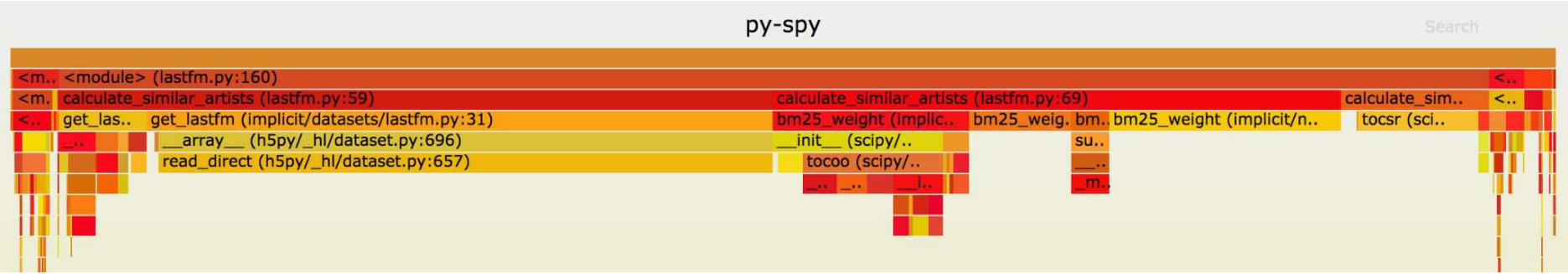
- Performance is limited by compute and memory bandwidth ceilings.
- Arithmetic Intensity determines the bottleneck:
  - low AI → memory-bound
  - high AI → compute-bound
- Modern GPUs are increasingly memory-bound.
- Optimize the bottleneck:
  - memory-bound → reduce memory traffic
  - compute-bound → improve compute efficiency
- Performance is about the interaction between algorithm and hardware.

Profiling

You cannot optimize what you do not measure

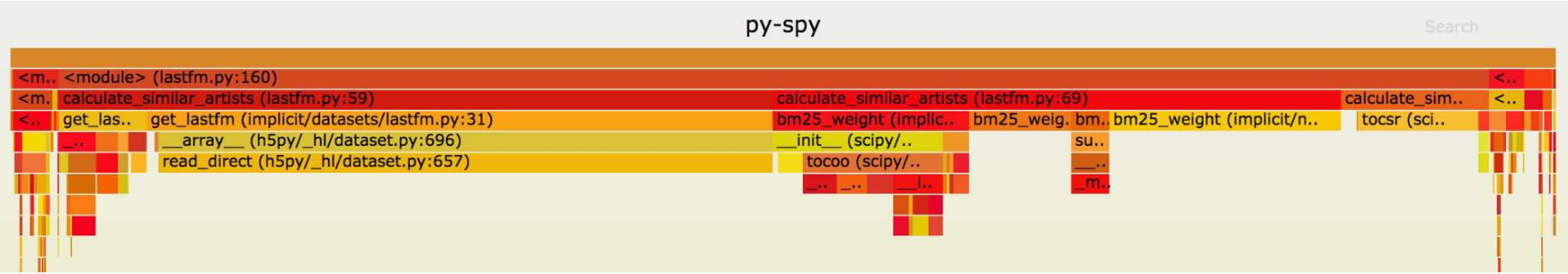
Profiling

- Measuring where time and resources are spent inside your program
- Helps find bottlenecks
- Two common approaches:
  - Sampling-based: periodically checks what is running
  - Event/tracing-based: records specific operations and timings



Profiling

- Profiling adds overhead, so don’t use profiled runs as final benchmark numbers
- For Python: cProfile, py-spy, ...
- For GPU workflow: PyTorch Profiler, NVIDIA Nsight Systems, Nsight Compute



PyTorch Profiler

- Easy to use: wrap your code in a context manager
- Aware of PyTorch runtime
- Exports traces viewable in TensorBoard or chrome://tracing
- Medium overhead: expect the profiled code to run 10–50% slower
- Best for "which PyTorch operation is expensive?"

```
with profile(activities=[ProfilerActivity.CPU], record_shapes=True) as prof:
    with record_function("model_inference"):
        model(inputs)
```

Key Features

- CPU/GPU activity tracking
- Operator-level performance metrics
- Memory consumption analysis
- Input shapes recording
- Stack trace examination
- Timeline visualization

Example

```
a = torch.randn((size, size), device=device, dtype=torch.float16)
b = torch.randn((size, size), device=device, dtype=torch.float16)
out = torch.empty_like(a)

torch.mm(a, b, out=out)
torch.cuda.synchronize()

with profile(
    activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
    record_shapes=True,
) as prof:
    with record_function("cpu_enqueues_cuda_kernels"):
        for _ in range(kernels):
            torch.mm(a, b, out=out)

    with record_function("cpu_waits_for_gpu"):
        torch.cuda.synchronize()

trace_path.parent.mkdir(exist_ok=True)
prof.export_chrome_trace(str(trace_path))
```

Example 2

```
a = torch.randn((size, size), device=device, dtype=torch.float16)
b = torch.randn((size, size), device=device, dtype=torch.float16)
out = torch.empty_like(a)

torch.mm(a, b, out=out)
_ = out.max().item()
torch.cuda.synchronize()

with profile(
    activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
    record_shapes=True,
) as prof:
    with record_function("cpu_enqueues_cuda_kernels_with_item"):
        for iteration in range(1, kernels + 1):
            torch.mm(a, b, out=out)

            if iteration == 3:
                with record_function("item_max_out_after_3_iterations"):
                    _ = out.max().item()

    with record_function("cpu_waits_for_gpu"):
        torch.cuda.synchronize()
```

Example 3

```
with profile(activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA], record_shapes=True) as prof:
    with record_function("GPU compute-bound: back-to-back matmuls"):
        for _ in range(7):
            torch.mm(a, b, out=out)
    torch.cuda.synchronize()

    with record_function("Host/CUDA API-bound: many tiny launches"):
        for _ in range(SMALL_KERNELS):
            small = small + 1
    torch.cuda.synchronize()

    with record_function("Synchronization-bound: CPU reads GPU value"):
        torch.mm(a, b, out=out)
        _ = out[0, 0].item()
    torch.cuda.synchronize()

    with record_function("Transfer-bound: host/device copies"):
        gpu_buffer = host.to(device, non_blocking=True)
        _ = gpu_buffer.to("cpu", non_blocking=True)
        torch.cuda.synchronize()

    with record_function("CPU/data-bound: GPU waits for Python"):
        time.sleep(CPU_SLEEP_MS / 1_000)
        torch.mm(a, b, out=out)
        torch.cuda.synchronize()

    with record_function("Overlap-bound: transfer and compute on separate streams"):
        with record_function("Copy stream: async pinned CPU -> GPU transfer"):
            with torch.cuda.stream(copy_stream):
                overlap_gpu_buffer.copy_(host, non_blocking=True)
        with record_function("Compute stream: matmuls run while copy engine is busy"):
            with torch.cuda.stream(compute_stream):
                for _ in range(4):
                    torch.mm(a, b, out=out)
        with record_function("Wait for transfer and compute streams"):
            copy_stream.synchronize()
            compute_stream.synchronize()
```

Typical Patterns

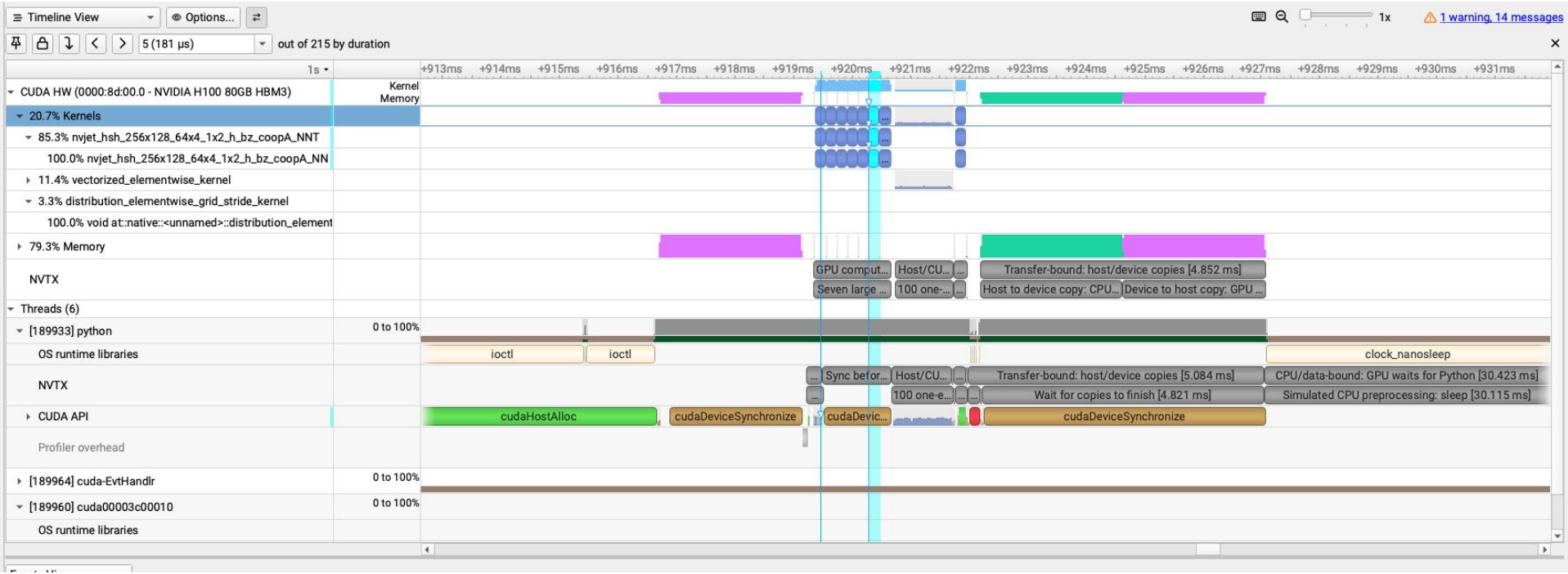
- GPU compute-bound: kernels run back-to-back and GPU is busy.
- Host/CUDA API-bound: many small launches and visible GPU gaps.
- Synchronization-bound: CPU waits for GPU, often from `.item()` or CPU reads.
- Transfer-bound: H2D/D2H copies dominate or block progress.
- CPU/data-bound: GPU waits for preprocessing, Python, or DataLoader work.

Nvidia Nsight Systems

- NVIDIA-grade timeline profiler - kernels, CUDA API, memcpy, NCCL, all on one timeline
- Low overhead
- Non-Intrusive
- Framework agnostic
- Best for "What is actually happening across CPU, CUDA runtime, GPU kernels, streams, and gaps over time?"

```
nsys profile <application>
    [application-arguments]
```

Example 4


Timeline View
Options...
1x
1 warning, 14 messages
5 (181 µs)
out of 215 by duration
1s
+913ms +914ms +915ms +916ms +917ms +918ms +919ms +920ms +921ms +922ms +923ms +924ms +925ms +926ms +927ms +928ms +929ms +930ms +931ms
CUDA HW (0000:8d:00.0 - NVIDIA H100 80GB HBM3)
Kernel
Memory
20.7% Kernels
85.3% nvjet_hsh_256x128_64x4_1x2_h_bz_coopA_NNT
100.0% nvjet_hsh_256x128_64x4_1x2_h_bz_coopA_NN
11.4% vectorized_elementwise_kernel
3.3% distribution_elementwise_grid_stride_kernel
100.0% void at::native::<unnamed>::distribution_element
79.3% Memory
NVTX
GPU comput...
Host/CU...
Transfer-bound: host/device copies [4.852 ms]
Seven large ...
100 one-...
Host to device copy: CPU...
Device to host copy: GPU ...
Threads (6)
[189933] python
0 to 100%
OS runtime libraries
ioctl
ioctl
clock_nanosleep
NVTX
Sync befor...
Host/CU...
Transfer-bound: host/device copies [5.084 ms]
CPU/data-bound: GPU waits for Python [30.423 ms]
100 one-e...
Wait for copies to finish [4.821 ms]
Simulated CPU preprocessing: sleep [30.115 ms]
CUDA API
cudaHostAlloc
cudaDeviceSynchronize
cudaDevic...
cudaDeviceSynchronize
Profiler overhead
[189964] cuda-EvtHandlr
0 to 100%
[189960] cuda00003c00010
0 to 100%
OS runtime libraries

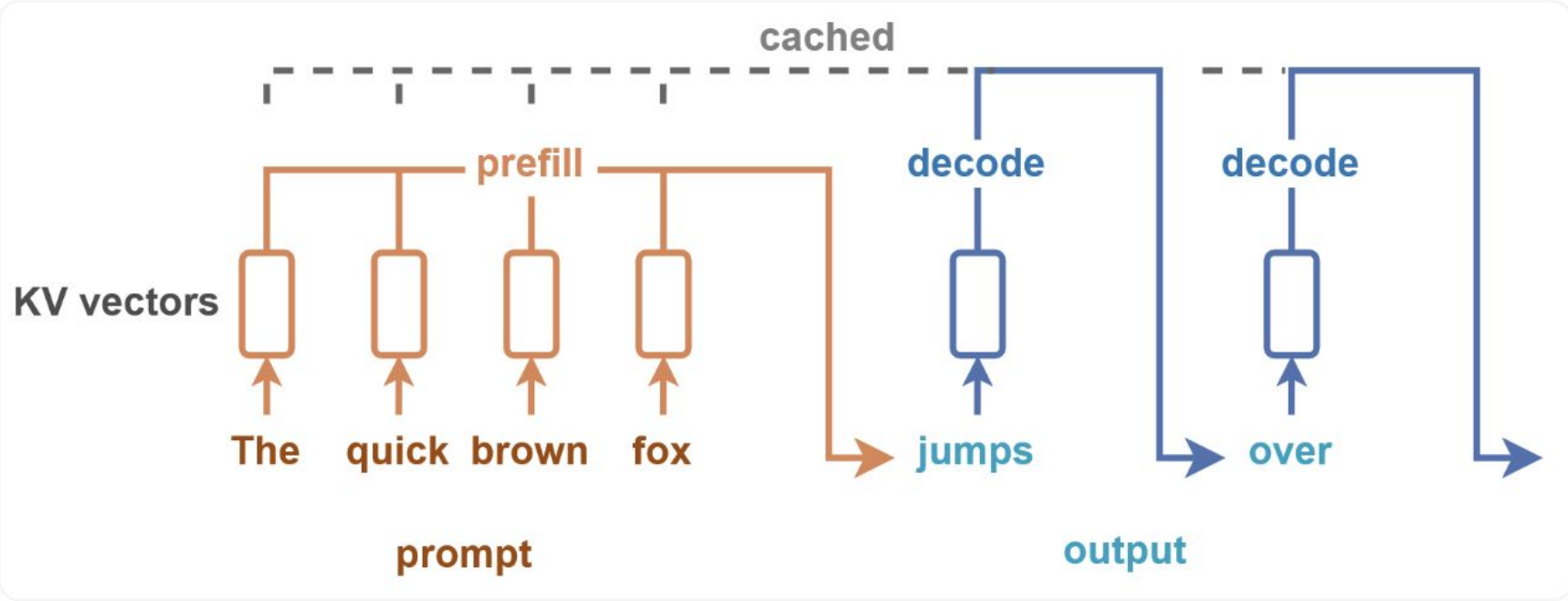
What to remember from part 1

- GPUs trade single-thread speed for massive parallelism; architecture reflects that.
- Modern AI workloads are often memory-bound, not compute-bound
- CUDA programming model: host launches kernels on a grid of thread blocks that map to SMs.
- Roofline model tells you whether your kernel is compute- or memory-bound, and what to optimize.
- Always measure

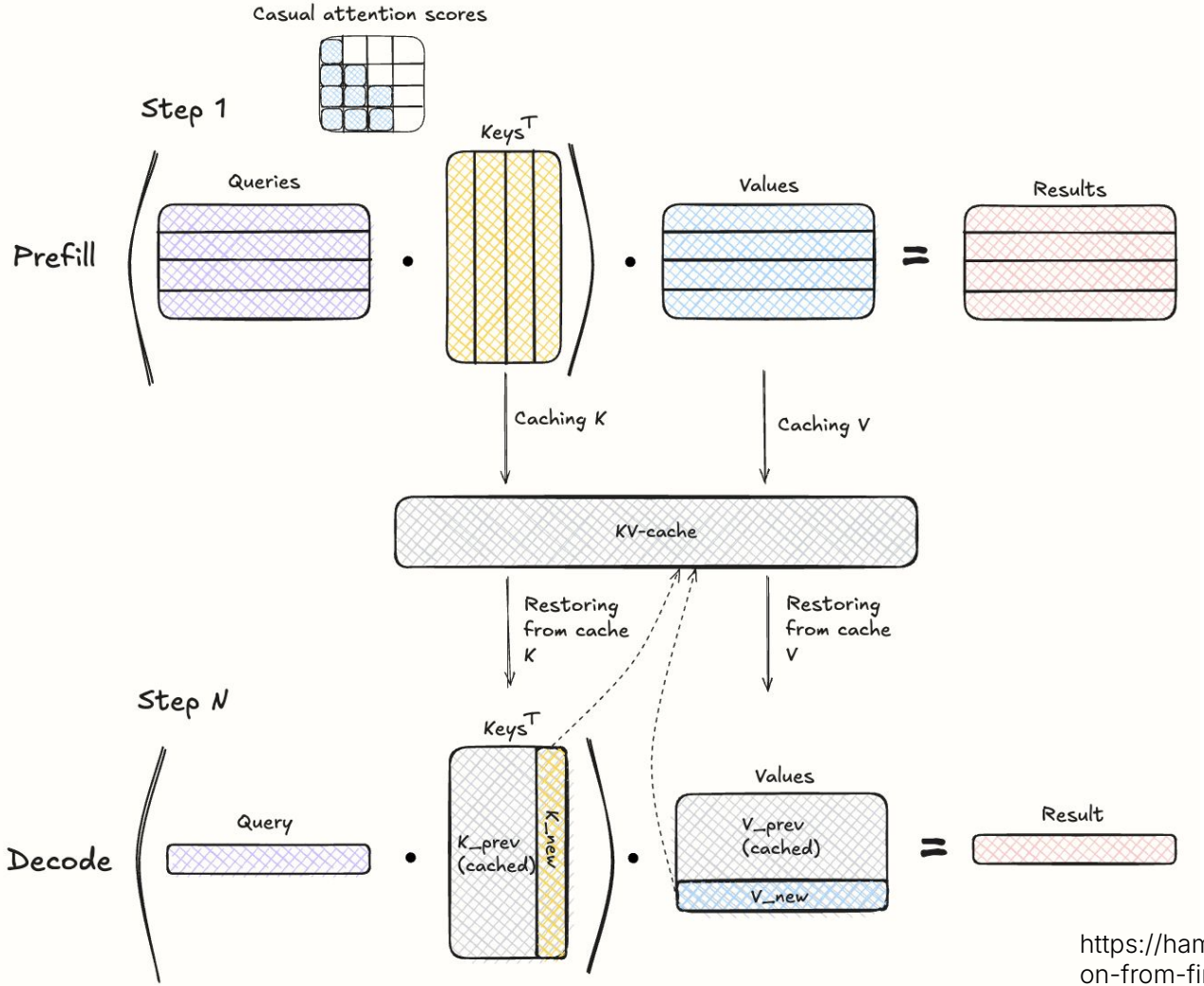
LLM Inference

Prefill is a single forward pass over the full prompt that builds the KV-cache and is typically compute-bound

Decode generates tokens one by one by reusing the KV-cache (and appending new KV values), and is typically memory-bound.



https://huggingface.co/blog/tngtech/llm-performance-prefill-decode-concurrent-requests



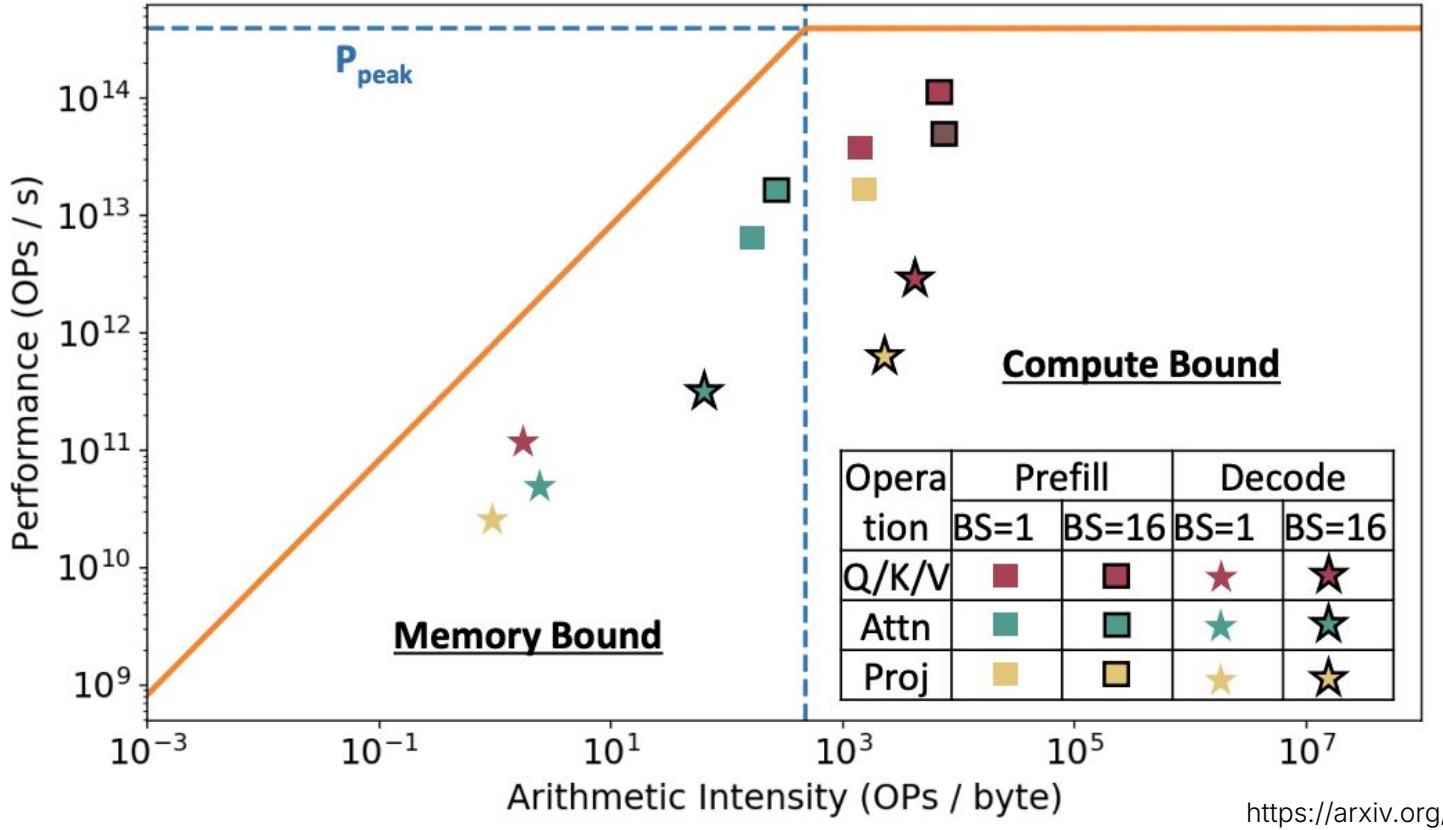
https://hamzaelshafie.bearblog.dev/paged-attention-from-first-principles-a-view-inside-vllm/

For Llama 3.3 70b, with 32k sequence length:

$$\begin{aligned}\text{KV cache size} = 2 &\times \text{num\_layers} \\ &\times \text{seq\_length} \\ &\times \text{num\_kv\_heads} \\ &\times \text{head\_dim} \\ &\times \text{bytes\_per\_value}\end{aligned}$$

$$\begin{aligned}&2 \times 80 \times 32{,}000 \times 8 \times 128 \times 2 \\ &= 10{,}485{,}760{,}000 \text{ bytes} \\ &= \frac{10{,}485{,}760{,}000}{1024^3} \text{ GiB} \\ &\approx 9.77 \text{ GiB}\end{aligned}$$

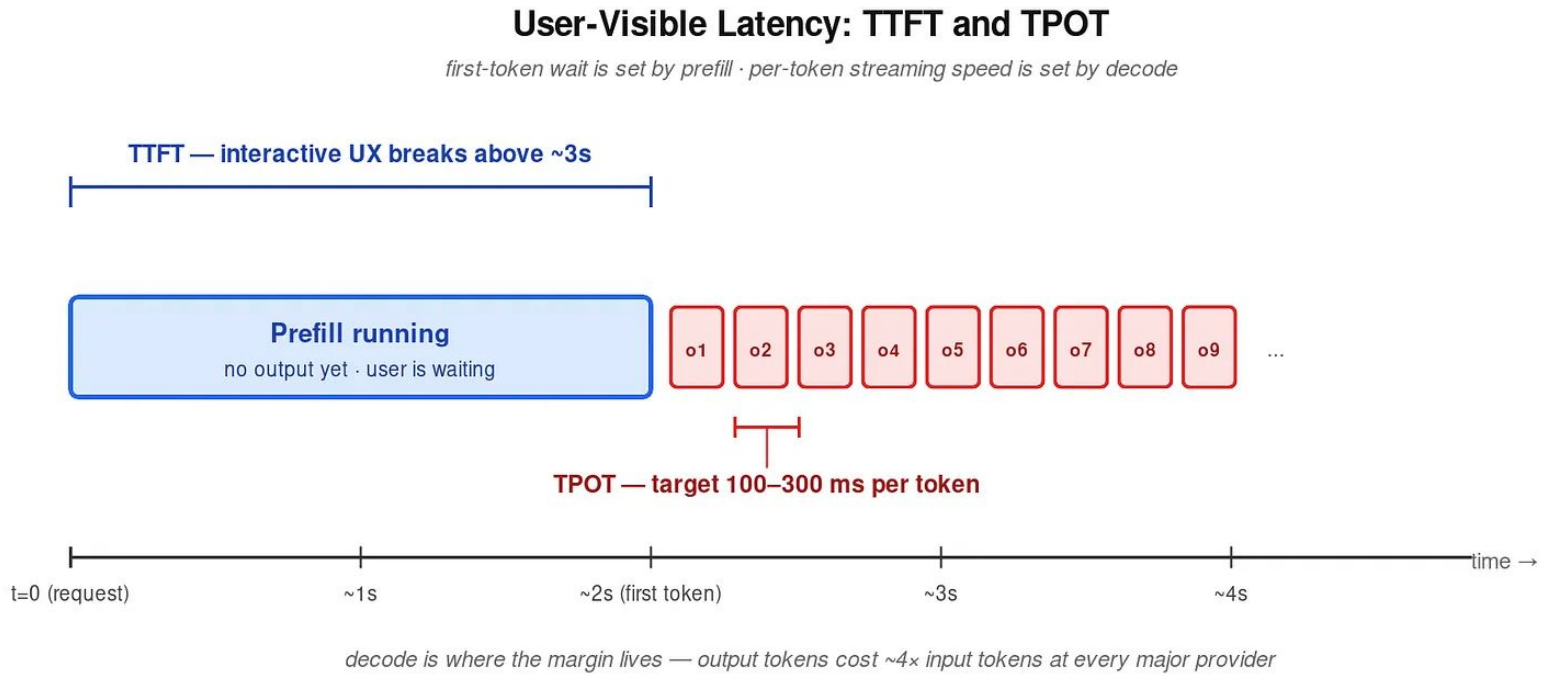
Prefill vs Decode



https://arxiv.org/pdf/2510.02675

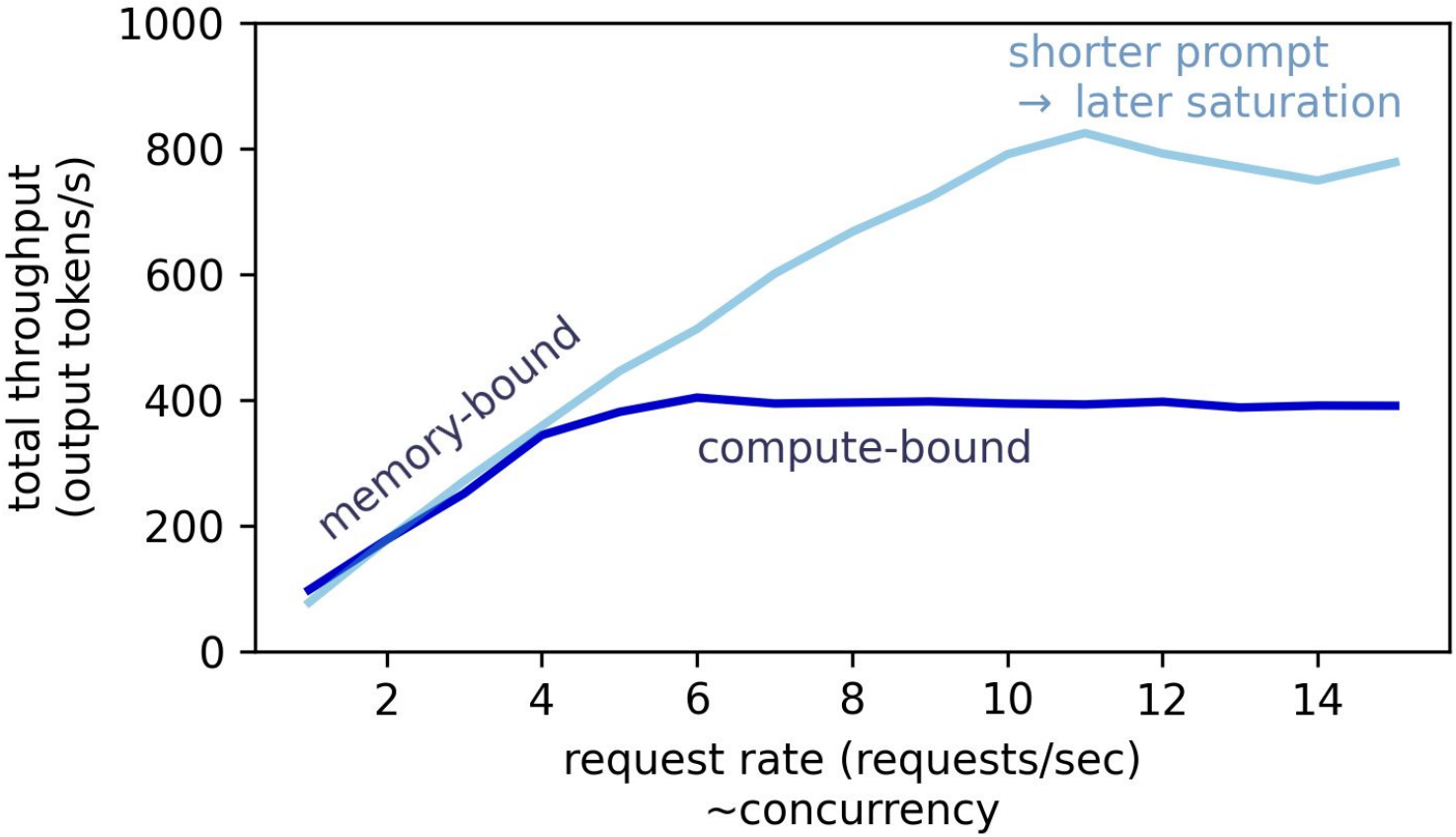
Metrics

TTFT, TPOT E2E latency Throughput (tok/sec) Cost



E2E latency ~= TTFT + num_output_tokens * TPOT

- Long prompts usually hurt TTFT first.
- Long output seq hurts E2E latency



https://huggingface.co/blog/tngtech/llm-performance-prefill-decode-concurrent-requests

Training vs Inference

	Training	Inference
Lifecycle	Expensive offline job, run during model development	Online service, runs continuously for users, for months / years
Batch shape	Large, regular, fixed	Small, dynamic, irregular
Dominant work	Forward + Backward	Prefill + many decode steps
Memory pressure	Activations + optimizer state	Weights + KV-cache
Common bottleneck	Compute / communication	Often decode-time memory bandwidth

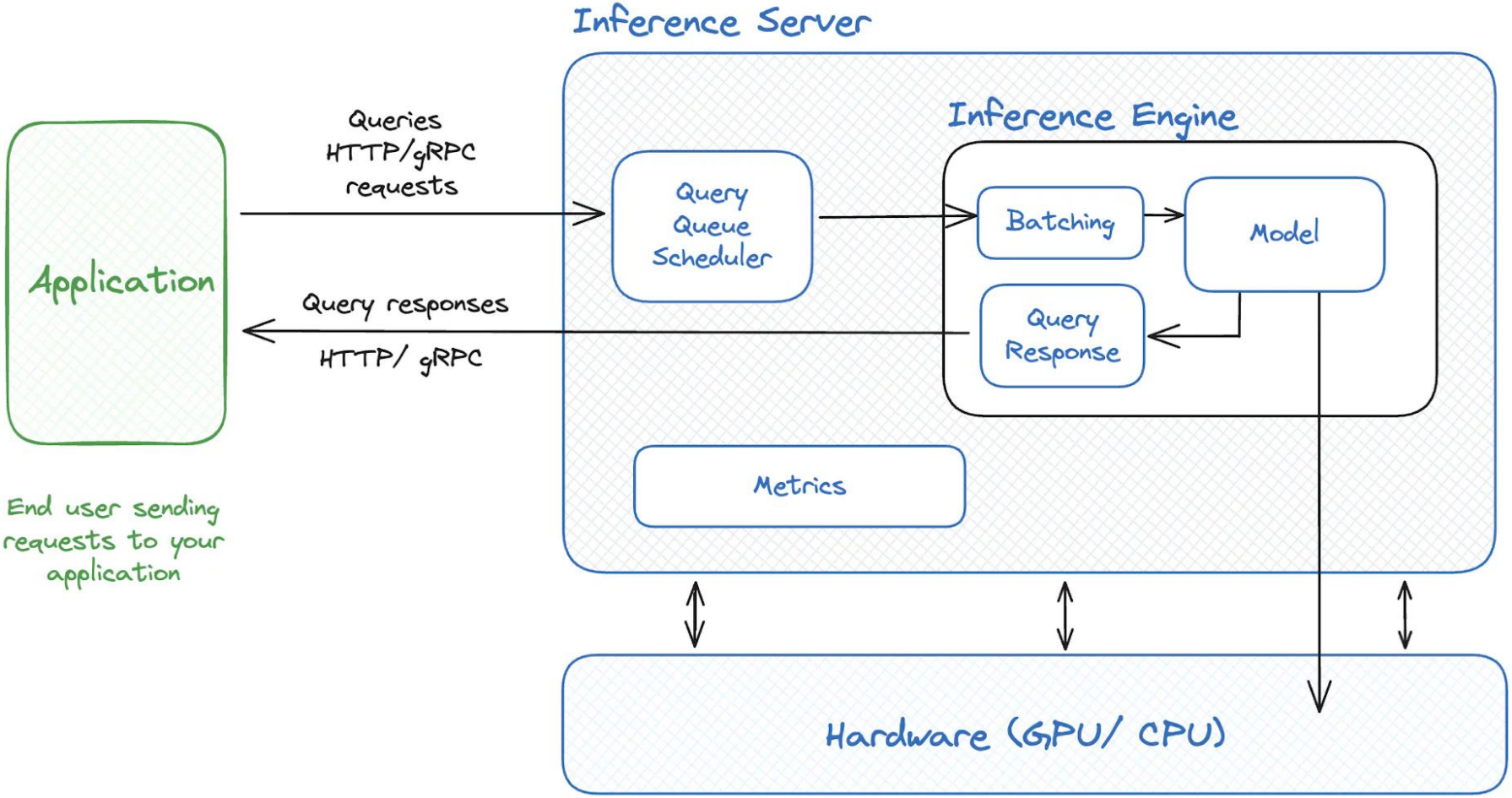
Inference Engine

Black box that takes input text, generation parameters and outputs the generated text.

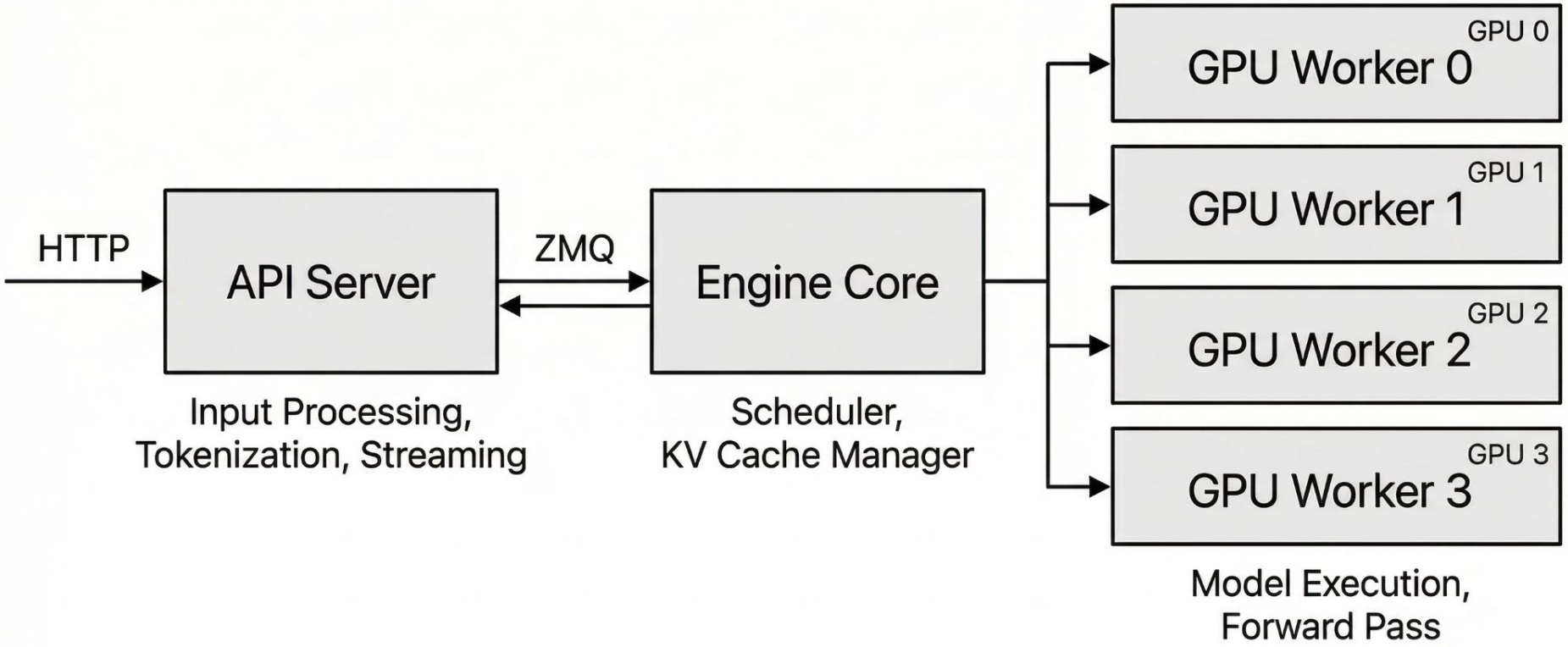
Fast and cost-effective.

Production ready.

A microservice, that can be optimized and scaled separately.


Inference Server
Inference Engine
Application
Queries
HTTP/gRPC
requests
Query
Queue
Scheduler
Batching
Model
Query responses
HTTP/ gRPC
Query
Response
Metrics
End user sending
requests to your
application
Hardware (GPU/ CPU)

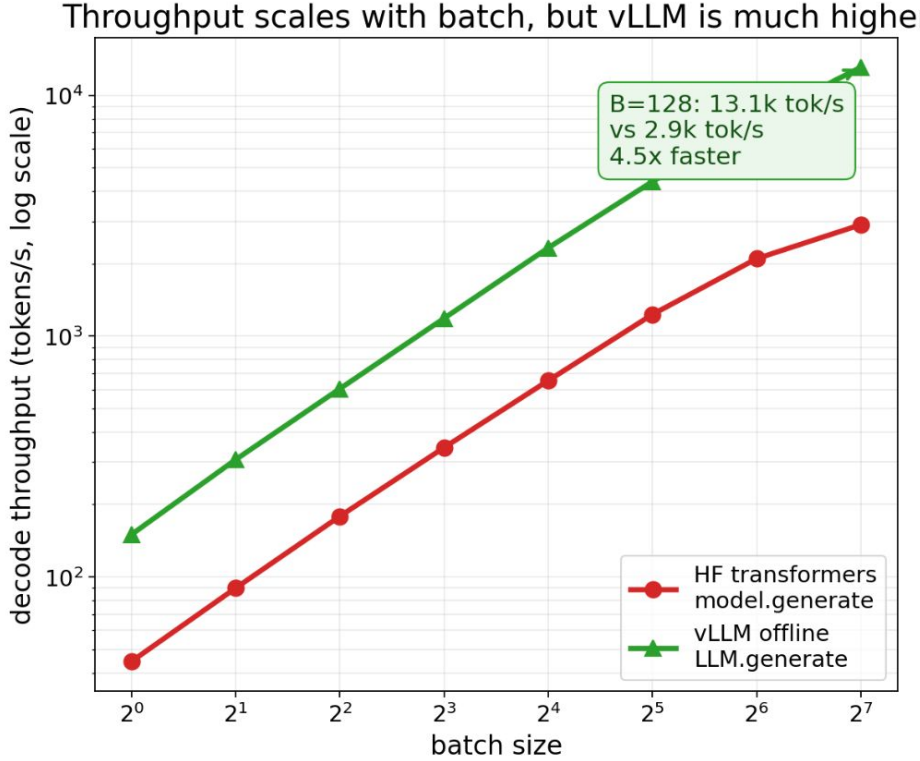
V1 Process Architecture (4 GPUs, TP=4)

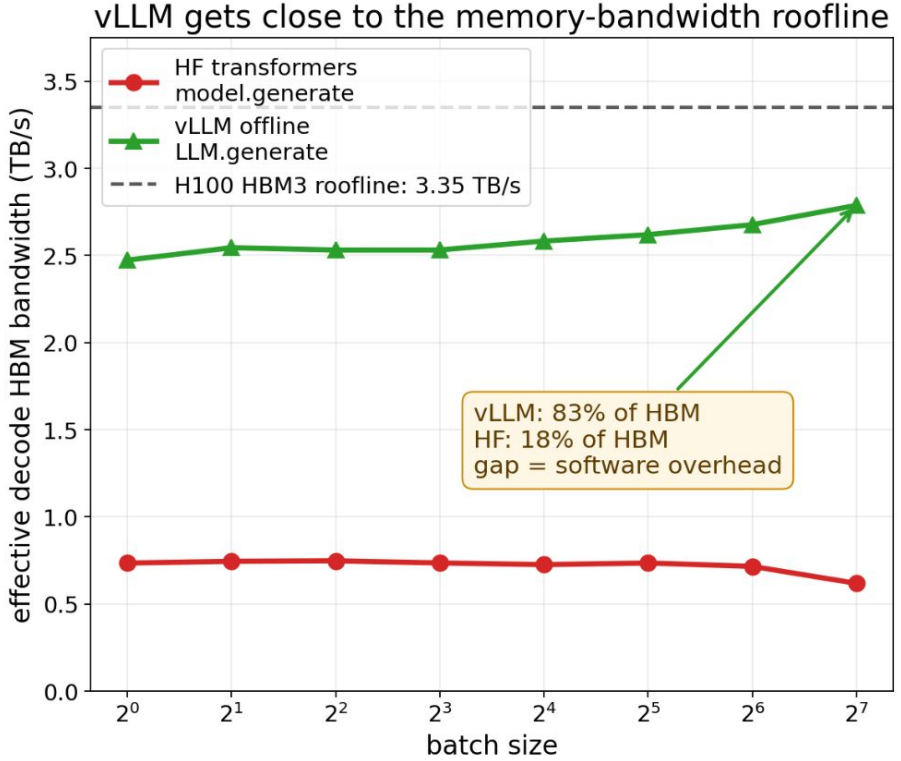


Total: 6 processes (1 API Server + 1 Engine Core + 4 GPU Workers)

vLLM converts batching into useful GPU memory bandwidth

Qwen3-8B BF16 on 1x H100 80GB | prompt=512 tokens | generate=128 tokens | greedy decode





Bandwidth is a roofline estimate: traffic = 16.4GB weights per decode step + batch x average context x 144KiB KV per token. Divide estimated traffic by measured wall time.

What to remember from part 2

- Inference has two phases: prefill is mostly compute-bound, decode is mostly memory-bound.
- KV-cache makes decoding fast enough to be practical, but turns serving into a memory-management problem.
- The serving metrics to watch are TTFT, TPOT, and throughput.
- Inference engines exist because production serving needs scheduling, batching, KV-cache management, and observability on top of the model itself.